Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

Taming Reentrancy with Auditable Language Design

Master Thesis

T. Richards

April 7, 2026

Advisors: Prof. Dr. Z. Su, Dr. H. Sun, Dr. C. Li
Department of Computer Science, ETH Zürich

Abstract

As decentralized finance protocols have grown to secure hundreds of billions of dollars in on-chain value, so too has the cost of failure: despite tens of millions spent annually on third-party audits, formal verification, and bug bounty programs, smart contract vulnerabilities continue to cause billions of dollars in losses. Among these, reentrancy – where an external call re-enters the calling contract before its state updates complete – remains one of the most common and damaging vulnerability classes. Because it is a purely structural property of the call graph rather than a semantic bug, it is uniquely suited to prevention through language design. We present a Rust-inspired smart contract language targeting the EVM that introduces two key safety mechanisms: 1. A compile-time effect system requiring developers to explicitly annotate state modifications. 2. Automatically generated reentrancy guards, with declarative annotations that allow developers to selectively re-enable specific reentrant call paths when required. Notably, the guarantees provided by the reentrancy guards allow the compiler to propagate side effect constraints even across reentrant call paths, which is not possible in any other language without sophisticated static analysis tools. The explicit effect annotations make storage mutation patterns machine-checked and visible at function boundaries, making smart contract code simpler to secure for developers, auditors, and bug bounty hunters. Together, these mechanisms render reentrancy vulnerabilities virtually impossible. We evaluate the practical impact of these features against real-world DeFi exploits (2021-2024) and find that 53/54 reentrancy incidents would have been prevented, the sole outlier being a compiler bug rather than a developer oversight.

Contents

Contents ii

1 Introduction 1

2 Background 3
2.1 EVM & Ethereum . . . 3
2.1.1 Overview . . . 3
2.1.2 World State . . . 3
2.1.3 Transactions . . . 4
2.1.4 Bytecode . . . 4
2.1.5 Execution . . . 5
2.2 Smart Contract Languages . . . 5
2.2.1 Solidity . . . 5
2.2.2 Vyper . . . 6
2.3 Smart Contract Vulnerabilities . . . 6
2.3.1 Reentrancy . . . 9
2.4 Security & Auditing . . . 10
2.5 Language Design Concepts . . . 11
2.5.1 Type systems and type safety . . . 11
2.5.2 Algebraic Data Types . . . 11
2.5.3 Safety by Default . . . 12
2.5.4 Side Effects . . . 12

3 Related Work 13
3.1 Security-Focused Languages . . . 13
3.1.1 Flint . . . 13
3.1.2 Scilla . . . 13
3.1.3 Move . . . 14
3.1.4 Idris . . . 14
3.1.5 Fe . . . 14
3.1.6 Remarks . . . 14
3.2 Reentrancy Protection Mechanisms . . . 15
3.2.1 Checks-Effects-Interactions . . . 15
3.2.2 Reentrancy guards . . . 15
3.2.3 Static Analysis . . . 16
3.2.4 Comparison with Our Approach . . . 16

4 Methodology **17**
4.1 Core Ideas . . . 17
4.1.1 Reentrancy Protection . . . 17
4.1.2 Side Effects . . . 18
4.1.3 Side Effects in External Calls . . . 20
4.1.4 Reentrancy as a Side Effect . . . 21
4.1.5 Algebraic Data Types . . . 22
4.2 Discussion . . . 22
4.2.1 Retrofitting Solidity . . . 22
4.2.2 Retrofitting Vyper . . . 23
4.2.3 Rust as a Base Language . . . 24

5 Implementation **25**
5.1 Pipeline . . . 25
5.2 Lexing, Parsing and Desugaring . . . 25
5.3 Linking . . . 26
5.4 Type Inference . . . 26
5.4.1 Notation . . . 27
5.4.2 Inference . . . 28
5.4.3 Method Resolutions . . . 28
5.5 Effect Checks . . . 29
5.6 Monomorphisation . . . 30
5.7 Code Generation . . . 33
5.7.1 Tuple types . . . 33
5.7.2 ADTs . . . 33
5.7.3 Pattern matching . . . 34

6 Evaluation **35**
6.1 Reentrancy Protection . . . 35
6.1.1 Performance Overhead . . . 36
6.2 Effect Annotations . . . 38
6.3 Algebraic Data Types . . . 40
6.3.1 Real-world example . . . 41

7 Conclusion **43**
7.1 Summary of Results . . . 43
7.2 Limitations . . . 44
7.3 Future Directions . . . 45
7.3.1 Adoption Road-Blocks . . . 45
7.3.2 Compiler Work . . . 45
7.3.3 User Studies . . . 45
7.3.4 Language Extensions . . . 46
7.4 Closing Remarks . . . 46

Bibliography **48**

Chapter 1

Introduction

Decentralised Finance (DeFi) is a multi-billion-dollar industry which consists of financial markets and services living on blockchains. Due to the unregulated nature of this space, it is not possible for conventional enforcement mechanisms to ensure parties adhere to deals they've made. Instead, the standard approach is to use programmatic **smart contract**s which automatically enforce the parameters of whatever deal is being made. Oftentimes, this is not a simple contract between two parties, but a complex self-managing system that can be interacted with by arbitrary users. Examples of such services are decentralized exchanges for converting between currencies, lending platforms for borrowing funds, and many other such applications. Due to the increasing complexity of such protocols, the importance of smart contract security is ever-growing.

The Ethereum blockchain [37] is one of the most popular smart contract ecosystems. In this space alone, hundreds of millions of dollars are lost due to smart contract vulnerabilities every year [24] [5] [34]. Due to the aforementioned unregulated nature and lack of enforcement mechanisms, there is no reliable way of recovering lost funds. As such, tens of millions of dollars are also spent every year on bug bounties, third party audits, and further security measures [24]. Since there is no centralized governing body, much of the ecosystem is based on trust – most projects are open-source, and users rely on the track records of developers and companies to make informed decisions about investments. Not every user is able to reason about the code or specification of every project, so they instead often rely on stamps of approval given by third-party audit companies. Because automatic tools are only capable of preventing around 8% of real-world exploits [7], manual audits dominate this space. Of course, audits are only as reliable as the code is readable – if a language obscures behaviour through implicit side effects or hidden control flow, even expert auditors can miss vulnerabilities. By requiring all code to be very explicit about what it's doing, as well as avoiding hidden behaviours, we aim to enhance the *auditability* of the language, making audits more thorough and reliable.

The Ethereum blockchain runs the Ethereum Virtual Machine (**EVM**), which dictates the execution model of smart contracts. Contracts are deployed as immutable bytecode, which is publicly available. However, the source code it was generated from may not be, although the vast majority of public projects publish their code as a way of building trust.

There are two widespread programming languages that are used to generate EVM bytecode. These are **Solidity** [17], an object-oriented language inspired by a variety of "curly-braced" languages, and **Vyper** [36], which is closely based on Python. Given their similarity with general-purpose programming languages, these languages were not initially designed with

built-in safety mechanisms to protect against EVM intricacies. This puts additional burden on developers to preemptively protect against vulnerabilities that are enabled not by their own code, but by the design of the EVM itself.

Designing a language with the peculiarities of the EVM in mind, as well as observing the importance of code auditability, has significant potential to prevent future exploits and reduce the risk involved with deploying new projects (or investing in them). We argue that moving security guarantees into the language design and type system, rather than relying on developer discipline, is the most reliable way of ensuring smart contract safety. Our three core features are motivated by this principle:

1. We specifically aim to prevent a vulnerability class called **reentrancy**, which will be thoroughly explained in chapters 2 and 4.
2. In order to improve auditability, we also require the developer to explicitly annotate hidden side effects, incentivizing a more "effect-oriented" approach to coding, as well as reducing the probability of unexpected interactions.
3. We offload responsibility from the developer to the compiler by providing an expressive type system, including **algebraic data types**.

Chapter 2

Background

2.1 EVM & Ethereum

In order to design a smart contract language for the Ethereum blockchain, it is a critical necessity to have a deep understanding of the EVM's inner workings. The full formal specification is detailed in the *Ethereum Yellowpaper* [37], but we will cover the necessary basics here. However, we will focus only on the EVM's execution model, leaving out the details of the Ethereum blockchain itself.

2.1.1 Overview

The EVM is a stack-based machine with a custom instruction set. Each instruction pops its arguments from the stack, then pushes its result (if there is one). Instructions consume **gas**, which is the cost measure of the EVM. Smart contracts may also allocate temporary values in memory, which does not persist between executions.

2.1.2 World State

The world state σ of the EVM is a mapping from addresses to account states. An address is simply a 20-byte string which can be used as a unique identifier for each account. There are two types of accounts: Externally Owned Accounts (EOAs), which are owned and operated by external users, and smart contracts, which contain bytecode to be executed.
An account state $\sigma[a]$ consists of the following four fields:

- Nonce $\sigma[a]_n$: For EOAs, incremented whenever a transaction is sent from this address. For contracts, incremented whenever they programmatically deploy a new contract (using the CREATE or CREATE2 opcodes).
- Balance $\sigma[a]_b$: A scalar value, denoting the amount of native currency (ETH) this account owns.
- Storage Root $\sigma[a]_s$: The hash of the storage state of this account. For our purposes, we can treat it as though it were storage state itself. Storage is a map from 256-bit integers to 256-bit storage values. Only contracts may contain storage values.
- Code Hash $\sigma[a]_c$: The hash of the bytecode of this account. Again, we will treat it equivalently to the bytecode itself. Only contracts may contain bytecode.

2.1.3 Transactions

There are several types of transactions, the details of which are not too important for us. Transactions are sent by external users, they can result either in a **message-call** or contract creation. When a transaction is sent, the sender's account nonce $\sigma[a]$ is incremented. They can further specify a maximum amount of **gas** to include, which is used to pay for the transaction and any resulting bytecode execution. If not enough gas is provided, the transaction will **revert**, meaning the state of the EVM is rolled back to what it was before the transaction began. The transaction may also include an ETH amount, which is transferred to the recipient account's balance $\sigma[r]_b$. For contract creation, no recipient address is given, and the ETH is instead transferred to the newly created contract. A transaction can also include **data**, which can either contain the constructor bytecode for contract creation, or be provided to a contract as input **calldata**.

2.1.4 Bytecode

The bytecode of a contract may consist of up to 24'576 bytes of code and data. The bytecode is read-only and can never be changed once a contract has been created. When a contract receives a **message-call**, the bytecode begins executing from its first byte. The standard calling convention is for the first four bytes of **calldata** to dictate the function to be executed, since a contract may have many different available functionalities. The remainder of calldata encodes the arguments to the specific function. Thus, the first segment of bytecode usually consists of *function dispatch*, i.e. decoding the input data and then jumping to the corresponding function's bytecode.

The instruction set is quite large, and contains many EVM-specific functionalities. A contract can read many different fields from block data, transaction data, or message data; or it can message-call further contracts using a variety of different call opcodes. Further, there are two opcodes related to contract creation, several for logging events (which are recorded in the transaction receipt). Importantly, the contract may also use the REVERT opcode, which rolls back to the state as it was before the message-call (but doesn't fully roll back the entire transaction).

When a new contract is created, its constructor bytecode must be provided. This constructor bytecode is directly executed by the EVM – when this execution terminates, the data it returns is written to the new account's bytecode $\sigma[a]_c$. The constructor may also already write values to the account's storage $\sigma[a]_s$.

There are several ways of working around the immutability of contract bytecode. The most common is to use a *proxy* contract. When a proxy contract is called, it forwards the call to another contract using the DELEGATECALL opcode. This opcode runs the bytecode associated with the recipient, but *uses the proxy contract's storage*. Thus, any state modifications are made in the proxy contract itself. By modifying which address the proxy points to, one can "update" the code it is running, since the new address that is being pointed to may contain different bytecode. While this is common practice, most serious projects have a time-lock to allow users to withdraw funds before an update is rolled out – this is a trust mechanism that prevents administrators from deploying a malicious implementation contract with no advance warning.

2.1.5 Execution

Execution usually begins with a **message-call**. Each message-call receives its own private **stack** and **memory**, which will be discarded when the message-call terminates. During execution, values may also be written to the contract's storage – however, if execution terminates exceptionally (e.g. by executing the REVERT opcode, or by running out of gas), then all state changes will be rolled back. A contract may also message-call into another contract, which will again have its own stack and memory. Importantly, if this second contract calls back into the first, the innermost message-call will have *its own stack and memory*, even though the contract is already mid-execution! This is called reentrancy, and often leads to concurrency bugs, such as read-write / write-read / write-write conflicts.

2.2 Smart Contract Languages

There are quite a few programming languages that can be compiled into EVM bytecode. However, by far the most common are **Solidity** and **Vyper**. Solidity has roughly 87% market share by total invested funds, and Vyper has another 8% [28]. Note also that not all projects publish their code, so it is not always clear what the source language was. Several other languages will be covered later in chapter 3.

2.2.1 Solidity

Solidity can be thought of as an object-oriented language that replaces objects with contracts. It is a relatively high-level, statically-typed language with a relatively complex type system. Since contracts behave like objects, they can inherit from one another. Many library contracts are declared `abstract` and declare `virtual` functions. Solidity even has multiple inheritance, and thus runs into typical issues like the diamond problem. User-defined types include `struct` and `enum`, but these are not object-like. Like in Java, Solidity distinguishes between primitive types (pass-by-value) and reference types. Primitive types include booleans, integers, addresses, function pointers, storage pointers, and enums. Arrays, strings, and structs are reference types, which must also specify their storage location – that is, they must declare if they are contained in `calldata`, `memory` or `storage`. Lastly, mappings are also reference types, but they can only be contained in storage.

Functions and Modifiers

Solidity functions can be declared using the `function` keyword. They may also specify visibility: `internal` functions can only be called by the contract itself, `external` functions only using message-call (i.e. from outside), and `public` functions can be called in both ways. Further, functions can declare mutability – `view` denotes that the function is read-only, i.e. it does not write state, and `pure` denotes that the function is mathematically pure, meaning it can not read from any non-constant values. Functions may also be declared `payable`, meaning they will not reject calls that include a non-zero amount of ETH along with the call.

Custom function *modifiers* can be declared using the `modifier` keyword. The modifier specifies code that will run *before* and *after* any function that includes it in its signature. For example, a typical use case is a `@nonReentrant` modifier, which simply adds a reentrancy guard that runs before any function that is declared as such.

As is typical in many object-oriented languages, functions may be *overloaded* and *overridden*. However, there is no operator overloading.

Remarks

Solidity was not originally designed with EVM intricacies in mind – it simply leaves all responsibility to the developer. While there are many smart contract libraries that implement protection mechanisms, this is all left to the developer. The most notable exception are overflow checks, which have been automatically generated since the release of Solidity v0.8 in 2020. The language also suffers from typical object-oriented issues – multiple inheritance, function overriding and other such features lead to confusing behaviour. They can make the control flow very difficult to follow.

2.2.2 Vyper

The Vyper language is based closely on Python, and was designed to be simple and intuitive – thus allowing both the code as well as the compiler to be as understandable as possible. It explicitly prioritizes language *readability* over simplicity in writing code. The language is intentionally quite limiting, in that it gives developers little low-level control over memory allocations, calldata and other such behaviours. The language also forbids unbounded loops and recursion, so that gas costs can always be bounded from above. Instead of inheritance, Vyper allows modularity through *composition*.

Functions and Modifiers

Similar to Solidity, Vyper functions can be declared with @external or @public visibility, with the default being internal. Vyper functions can also be declared as @view, @pure and @payable, with similar implications as in Solidity. Lastly, Vyper has built-in reentrancy protection mechanisms, allowing the developer to declare a function to be @nonreentrant. Alternatively, the line # pragma nonreentrancy on can be included to disable reentrant calls altogether.

Remarks

In terms of language design, Vyper is already a massive step up over Solidity. It prioritizes simplicity and auditability over developer convenience, and deliberately omits more complex features such as inheritance and function overloading as a safety measure. However, while it does feature a few built-in protection mechanisms, it is still up to the developer to opt in, in most cases. It also gives little control over memory allocations, and due to a lack of optimizations this often results in a large memory footprint.

2.3 Smart Contract Vulnerabilities

There is no community-wide standard taxonomy of smart contract vulnerabilities [31]. Especially when analyzing real-world exploits, multiple different bugs or attack vectors can be combined, meaning there is no clear category to be assigned. Nonetheless, most cases do have a relatively clear "main" category that they fit into. Throughout this thesis, we will be using a dataset of exploits collected by DeFiHackLabs [11]. While their list does contain more cases, we excluded those that were not programmatic in nature (e.g. key compromise), or those that were due to off-chain bugs occurring outside the EVM (e.g. bridge exploits).

Finally, we only included data from 2021-2024, as the data for previous and subsequent years was not (yet) conclusively collected. Thus, we obtained a sample set of 549 classifiable exploits for our analyses. The two main data points we are interested in, are the **prevalence** and **impact** of each vulnerability class. That is, how often is each vulnerability exploited, and how severe are the exploits? We measure the impact by financial damage caused – not necessarily the amount successfully stolen. We do not attempt to assess reputational damage or other such cost measures. We have categorized the exploits into ten vulnerability classes:

1. Business Logic – Flaws in the intended behaviour of the contract, the code behaves correctly but the specification is susceptible.
2. Price Manipulation – The contract relies on an external oracle contract to determine an exchange rate, but the oracle is not reliable enough (i.e. can be manipulated).
3. Access Control – Missing or incorrect permissions regarding who can call sensitive functions.
4. Reentrancy – An external call allows a malicious contract to re-enter the calling contract mid-execution, leading to read / write conflicts.
5. Precision Loss – Rounding errors in integer (or fixed-point decimal) arithmetic that accumulate or can be otherwise exploited to drain funds.
6. Input Validation – Failure to check validity of user-supplied arguments.
7. Token Design – Vulnerabilities arising from non-standard or malicious token behaviour that was not accounted for.
8. Arbitrary Call – Allowing an untrusted external call with attacker-controlled target or calldata.
9. Slippage – Insufficient protection against unfavorable price movements between transaction submission and execution.
10. Other – Miscellaneous vulnerabilities that do not cleanly fit into the above categories.

Even if a vulnerability is exceedingly common, if it rarely results in large damages, then it may not be too critical to protect against. However, if a vulnerability is very rare, but on average results in very high damages, then this is definitely a concern. The rarity means that developers will not be so familiar with protecting against such a vulnerability, despite the high risk involved. Finally, if we simply want to minimize potential damages, then preventing vulnerabilities in descending order of total lost funds would suffice. Figure 2.1 compares the **prevalence** of each vulnerability over the four-year period, while figure 2.2 compares their **impact**. The quantitative results are summarized in table 2.1.

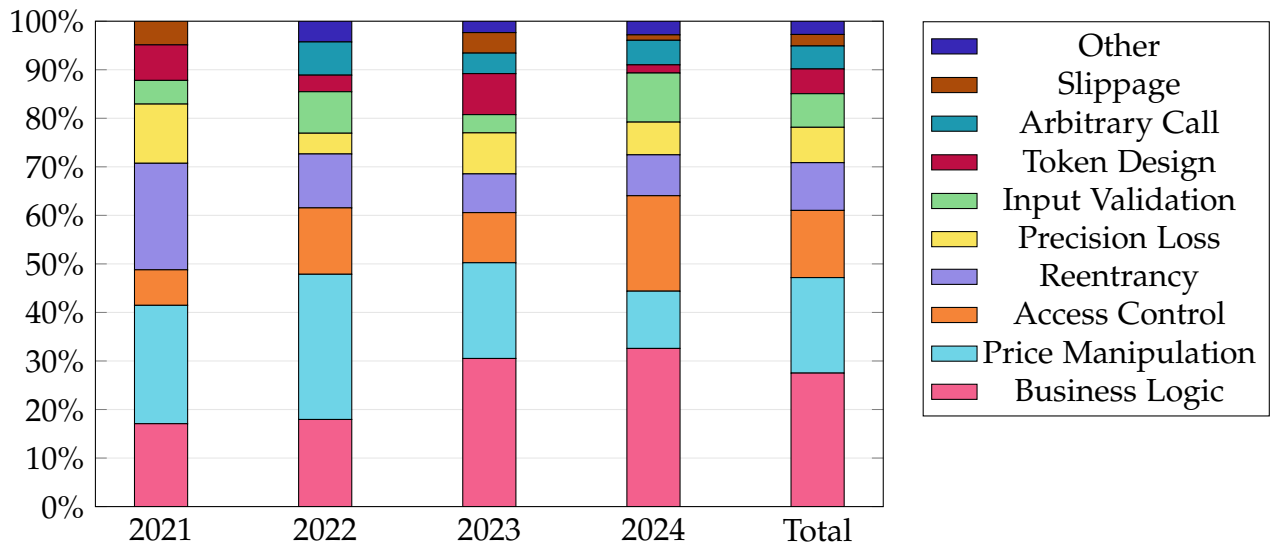

Figure 2.1: Smart contract vulnerabilities by count, 2021–2024.

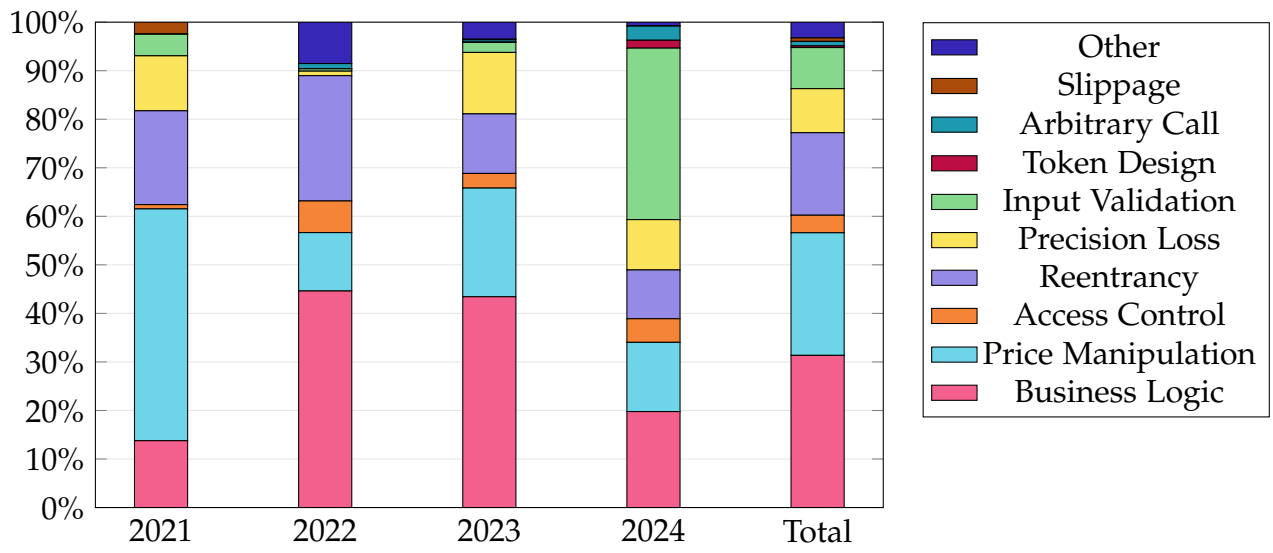

Figure 2.2: Smart contract vulnerabilities by total funds lost, 2021–2024.

As we can see, some vulnerability types generally lead to larger damages. We can compare by average funds lost per exploit, although due to the vast differences in exploit size, large outliers can strongly influence the average. Therefore, we also consider the median exploit size.

We can clearly see that reentrancy is by far the most impactful vulnerability class, both by average and median exploit size. Of course, it always depends on the analyzed time period – in 2022, the term **read-only reentrancy** was coined [6] to describe a new kind of reentrancy exploit. Often, when novel attack patterns are discovered, there is a subsequent flurry of exploits taking advantage of vulnerable protocols. However, given our dataset covers an entire four-year period, which corresponds roughly 40% of Ethereum's entire lifespan, it should still be relatively meaningful. Moreover, including older data might be misleading, since some kinds of vulnerabilities such as overflows simply do not happen frequently anymore, either due to improved tooling or widespread knowledge of how to mitigate them.

Class	Occurrences	Total Funds Lost	Average Loss	Median Loss
Business Logic	151	$452 million	$3.47 million	$57k
Price Manipulation	108	$422 million	$3.91 million	$88k
Access Control	76	$60.5 million	$0.796 million	$78k
Reentrancy	54	$284 million	$5.26 million	$725k
Precision Loss	40	$151.7 million	$3.79 million	$200k
Input Validation	38	$141.5 million	$3.72 million	$150k
Token Design	28	$5.63 million	$0.201 million	$14k
Arbitrary Call	26	$15.7 million	$0.604 million	$114k
Slippage	13	$11.75 million	$0.904 million	$76k
Other	15	$54.4 million	$3.63 million	$68k

Table 2.1: Vulnerability classes by prevalence and impact

2.3.1 Reentrancy

Unfortunately, reentrancy has been a well-known issue for a long time. The first ever reentrancy exploit, known as **the DAO hack** [13], garnered incredible attention at the time. In fact, this resulted in the only ever retroactive hard-fork in Ethereum's history! That means the majority of the Ethereum community agreed to roll back the blockchain to a block *before* the hack occurred, and agreed to modify the contract's state so that it was no longer exploitable. This was very controversial at the time, and led to a literal divide in the community – the original blockchain was still continued and has become known as **Ethereum Classic**, albeit with a smaller following.

Since then, reentrancy has continued to plague projects deployed on the Ethereum blockchain. It is one of the most well-known vulnerabilities, especially due to the fact it is unique to the EVM. Most developers know about it and try to protect against it, but clearly not sufficiently. It is also conceptually easy to distinguish from other vulnerabilities, by asking a simple question: Could the same state changes have been made with two (or more) *sequential* calls, instead of two *nested* ones? If not, then reentrancy must have been at play.

To make matters worse, the vast majority of contracts do not even require reentrant behaviour to operate correctly. Developers are simply not adequately protecting against reentrant calls, even though most projects could just reject every reentrant call by default. As we have seen, reentrancy makes up around 10% of vulnerability exploits today, and 17% of lost funds. The natural question to ask is *why*? How can such a well-known and prevalent vulnerability still be an issue ten years after its first discovery? Clearly, there must be a lack of adequate protection mechanisms in available tools. We will discuss these mechanisms in more detail in chapter 4.

Read-only Reentrancy

We briefly mentioned **read-only reentrancy** as a novel type of attack pattern. What makes it unusual is that the exploited contract is not usually reentered. In most cases, the main

contract is relying on an external **oracle** contract to determine some value, often an exchange rate between currencies. The attacker is then able to influence the oracle in some way, for instance by trading or depositing funds. The oracle then calls back to the attacker, who can now call the victim contract. Due to the oracle still being mid-execution, when the victim contract reads values from the oracle, they may be in an inconsistent state. In some cases, this can then allow the victim contract to be drained of funds. While it is an interesting attack pattern, there's nothing special about the reentrancy itself here – the bug originates from a write-read conflict in the oracle contract.

2.4 Security & Auditing

Having seen the variety of vulnerabilities and attack patterns, it becomes clear that identifying them requires significant expertise. Developers cannot be expected to anticipate every possible exploit, especially those requiring unusual or novel attack patterns. Similarly, users of protocols are even less likely to be able to reason about the security of code. Hence, both groups rely on third-party auditors, who on the one hand provide developers with private bug reports so they can amend their code before deployment, and on the other hand publish an audit report detailing the findings, so that users can make more educated guesses about the reliability of a new project.

Furthermore, automatic tools are not adequate enough to provide useful findings in most cases. Thus, the ecosystem heavily relies on auditors as the primary mechanism for catching vulnerabilities before deployment. In addition, bug bounty programs are quite common, either before deployment as a sort of "crowd-funded" audit, or more frequently to incentivize responsible disclosure so that vulnerable contracts can be updated or retired. The downside of such bug bounties is that most contributors will not spend much time diving deep into the code, instead submitting a vast amount of surface-level issues to optimize for quantity of bugs over quality. By contrast, auditors are compensated for their time, not their findings, letting them gain a much deeper understanding of the codebase and its potential issues.

Given the reliance on human feedback for vulnerability detection, it becomes important to optimize code for *auditability*. This is one of the driving factors behind Vyper's design – simplicity in the language design allows auditors to quickly understand what is going on. This is not necessarily the case in Solidity, since inheritance and function overloading / overriding means it is not immediately clear what code is being executed when a function is called. Therefore, an auditor must first deal with the nebulous language design to even understand what code is being executed, let alone whether or not it is correct. Hence, the importance of language design becomes clear: By optimizing code for *auditability*, we can accelerate and improve the audit process, making it faster, cheaper, and more effective at finding bugs.

Note that code *readability* is not necessarily the same thing – readability implies the ease of inferring what code does. However, auditors want to do the opposite: They want to be sure about what the code *does not* do, i.e. rule out vulnerabilities. This can be done for example by making side effects explicit, instead of expecting the auditor to always be able to infer when some implicit effect is occurring. Alternatively, the type system of a language can enforce certain properties, for example that the failure case of an external call must be correctly handled. Again, this shifts the burden away from developer / auditor discipline, and onto the compiler.

2.5 Language Design Concepts

2.5.1 Type systems and type safety

A type system defines a set of rules for assigning types to program expressions, which enables the compiler to reject problematic programs at compile time. In principle, this means if we can define a type system that rules out certain kinds of vulnerabilities, then those vulnerabilities become impossible in our language. In the context of smart contracts, type safety is particularly important, since it allows bugs to be caught before deployment. Another important factor is the *expressivity* of the type system – the more expressive a type system is, the larger the variety of concepts that can be natively modelled in it. This is an attractive feature, since it means developers don't have to implement their own functionality, and it again provides the opportunity for bugs to be caught at compile time. Note that *expressivity* does not necessarily imply *complexity*.

Type Inference

Forcing a developer to write out the type of every expression explicitly can become very tedious. Thus, most languages *infer* the types of (sub-)expressions automatically by following the rules of their type system. This can be taken a step further, by not only inferring the expressions, but for example also the types of local variables and functions. If a type cannot be inferred by the compiler, it will simply require the user to specify the type explicitly. Type inference is a mechanism to make a language more convenient for developers, especially if its type system is difficult to satisfy. Hindley-Milner inference [23] [27], or more specifically Algorithm W [10], is a sound type inference algorithm typically used for functional programming languages, but can also be applied to many other type systems.

2.5.2 Algebraic Data Types

Algebraic Data Types are commonly used in functional programming languages, but have also made their way into more mainstream programming languages such as Python and Rust. Rust's `enum` is not a traditional enumeration type, but instead an algebraic data type. ADTs typically allow composing types in two distinct ways: As a **product**, which means combining types as in traditional structs; or as a **sum**, which represents one of several variants, as in traditional enums. The key point is that these products and sums can be combined in arbitrary ways. It also means that every traditional struct or enum is by definition also an ADT which only uses one of the two composition rules. The most common examples of ADTs are the `Maybe` / `Option` type for nullable / optional values, and the `Result` type for error handling.

Option The `Option` type is defined as the sum of an empty type, and an arbitrary type T. This means it can either contain a value of type T, or nothing – this is in a sense equivalent to allowing a null value as in Java, but allows the compiler to enforce that every `Option` type must first be checked to be non-null before using it. In Rust, it is defined as:

```
enum Option<T> {
  Some(T),
  None,
}
```

Result The `Result` type is defined as the sum of two different arbitrary types T and E. T represents the result of a successful computation, whereas E holds information about the error case, for example an error message. Therefore, a `Result` natively represents the *result* of a potentially failing computation. Alternatively, it can be used to represent a computation that can simply return one of two different types of results. The type is defined as follows:

```
enum Result<T, E> {
  Ok(T),
  Err(E),
}
```

Pattern Matching What makes ADTs so powerful is that they can model alternatives that may contain further data. For example, they are extremely useful when modelling abstract syntax trees. There are many different kinds of program expressions, each of which contains many fields. This can be directly modelled with ADTs, as opposed to having to declare one struct for each expression type, as well as a separate enum to distinguish them. They are also a perfect choice for modelling state machines, which are quite relevant for smart contract development.

Usually, ADTs are combined with pattern matching, which is in essence a more powerful version of a `switch` statement. Pattern matching allows developers to distinguish between the various representations of sum types, which may be arbitrarily nested. Importantly, it is possible for the compiler to check that the match is **exhaustive**, meaning that every possible case has been covered.

2.5.3 Safety by Default

This is a design philosophy in language design which dictates that safe behaviour must be the default. Any unsafe behaviour requires the developer to explicitly opt in. For example, in Rust, all local variables are immutable by default, and can be explicitly declared mutable with the `mut` keyword. By contrast, Solidity functions are mutable by default, and can be explicitly declared to be `view` or `pure` to restrict them – exactly the opposite approach.

2.5.4 Side Effects

Side Effect is a catch-all term which essentially encompasses anything that isn't explicitly contained in a function's signature. A function is *pure* if it has no side effects, meaning its output depends only on the arguments passed. For example, reading from transaction data is a side effect, since it is not explicitly provided as an argument. Logging an event is also a side effect, since it is externally visible but not part of the function's result type. For our purposes, we will mostly be focusing on state modifications, i.e. when a contract writes to storage.

There are many different systems of handling side effects, and there is a rich theoretical background on effect systems in functional programming languages. However, in imperative languages, the concepts are much less formalised. One example is the throws keyword in Java, which specifies in what ways a function might fail. Any caller of such a function is required to handle every declared failure case. In many cases, it is more important to make the side effect *explicit* rather than mathematically modelling it in a perfectly accurate way. Annotations are the prototypical way of doing this, essentially by including the effects in the function signature, or otherwise marking areas in the code that may cause side effects.

Chapter 3

Related Work

3.1 Security-Focused Languages

There have been many previous efforts to design smart contract languages, or to adapt general-purpose languages to compile to EVM bytecode. Of course, many remain experimental or are simply outdated, but this is not too surprising. Several languages are also purposely restrictive in their domain: Zinc [26] is a language solely intended for writing zero-knowledge circuits and contracts based on zero-knowledge proofs, and Lira [1] [12] is a domain-specific language for financial contracts, intended for those with a background in finance rather than programming.

3.1.1 Flint

The Flint programming language [32] is a safety-oriented language based on Swift, with several notable security mechanisms. Its **caller capabilities** dictate who can call which external functions, as a built-in access control mechanism. In addition, each state-modifying function must declare the storage variables it may modify. Flint also provides linear asset types, allowing token balances to be modelled directly by the type system. This prevents double-counting or double-spending bugs at compile time. Since the language was originally designed, it has been extended so as to be *formally verifiable* [21], by converting Flint code to the Boogie verification language. Despite the features pioneered by Flint being quite powerful, it has not been widely used. Moreover, Flint does not provide any built-in mechanisms to deal with reentrancy directly – instead, it simply assumes that external calls may cause any state changes that do not violate a contract's declared invariants (if verification is enabled). This significantly hinders verification across call boundaries – meaning it becomes quite difficult to verify the absence of reentrancy bugs.

3.1.2 Scilla

Scilla [33] (**S**mart **C**ontract **I**ntermediate-**L**evel **LA**nguage) is an intermediate representation for smart contracts for the Zilliqa blockchain. Contracts are modelled as automata, meaning computations are modelled via state transitions. Further, computation and communication are conceptually separated, meaning external calls must be in tail position, i.e. they can only be executed as the last operation. The type system ensures certain properties about contract transitions, making Scilla programs more easily automatically verifiable. The restriction of external calls to tail position prevents reentrancy bugs entirely, but also severely restricts the interactivity of contracts.

3.1.3 Move

Move [2] is a resource-oriented language originally designed by Facebook / Meta for the Libra blockchain, but has since been discontinued. Some community efforts have been made to port it to other platforms such as Aptos. Similar to Flint, the language models assets with linear types, preventing duplication of token amounts. Further, it provides native access control mechanisms, again by defining custom *resource types*. Unfortunately, the properties enforced by the language require the bytecode to be *verified* before deployment, to ensure that all relevant safety properties hold. Due to this restriction, applying Move to the EVM would be a futile endeavour, since the properties assumed by the language would not hold in any other environment.

3.1.4 Idris

Idris [3] [4] is a general-purpose functional programming language with dependent types. It also features an experimental backend for compiling to EVM bytecode [30]. This backend makes use of the effect system built into the language to model side effects such as ETH transfers and storage mutations. Further, they build cryptographic commitments into the type system, so that hidden information is guaranteed not to be revealed early. The functional nature of the language of course means all side effects must be explicitly stated, but the feature set of the custom backend is quite limiting. Since external calls are not yet supported, reentrancy vulnerabilities are of course prevented by default.

3.1.5 Fe

Fe [14] was originally a re-implementation of the Vyper compiler in Rust. Since its initial release, more and more Rust-like features were added, resulting in a significant amount of technical debt due to the design diverging from the initial goals. In March 2026, the language was redesigned from the ground up [16] to be much more closely based on Rust, to make it a more compelling alternative to Solidity and Vyper. Due to its recent overhaul, the language is still experimental and not yet production-ready. This redesign includes an effect system, requiring functions to explicitly declare the resources they interact with, such as storage, the execution context, and so on. Apart from the effects, the type system is very similar to Rust, including ADTs and parametric polymorphism. Further, the language requires explicit definitions of *messages*, which represent the external interface of a contract. A contract can declare a *receive* block to declare the implementations for each message type. The functionality can either be implemented directly there, or be defined for example on a user-defined type to conceptually separate the contract from its functionality. So far, the language has not yet implemented any cross-contract features beyond basic external calls, much less any built-in reentrancy protection mechanisms. However, given the recent flurry of activity, this may well change in the near future.

3.1.6 Remarks

Across all reviewed languages, no existing languages targeting EVM have sufficient protection mechanisms to prevent reentrancy vulnerabilities – the most impactful vulnerability class in our dataset. Fe is the closest in spirit, featuring both an expressive Rust-like type system and explicit effect declarations. Notably, its redesign occurred during the writing of this thesis, suggesting these are natural abstractions for the problem space. However, Fe

still lacks reentrancy protection mechanisms, leaving this as the primary remaining gap our work addresses.

3.2 Reentrancy Protection Mechanisms

Despite reentrancy being one of the most prevalent and costly classes of smart contract vulnerabilities, existing protection mechanisms remain largely inadequate. We survey the most widely used approaches and discuss their shortcomings

3.2.1 Checks-Effects-Interactions

The checks-effects-interactions (CEI) pattern [15] is the most widely recommended coding convention for preventing reentrancy. It dictates that a function should be structured in three parts: First, *check* all preconditions and validate any inputs; second, update internal state (*effects*); and third, perform any external calls (*interactions*). This prevents reentrancy vulnerabilities by ensuring that state is always fully updated before an external call, meaning the state will be in a consistent state during the external calls.

The CEI pattern has several shortcomings. First of all, it is purely a coding convention, and is not enforced by the compiler. Therefore, it relies purely on developer (or auditor) discipline. Clearly, this is not sufficient in practice, as demonstrated by our vulnerability dataset. More importantly, the pattern cannot always be applied – sometimes, a contract requires several interactions to happen, with checks or state updates in between. Lastly, the pattern does not exclude all possible reentrancy bugs, it merely guarantees that the contract's state is not inconsistent during external calls.

3.2.2 Reentrancy guards

Reentrancy guards are by far the most common protection mechanism. They are defined adding annotations to the entrypoints of a contract. The standard implementation writes a boolean flag to storage when an annotated entrypoint is entered, and clears it when execution terminates. Whenever the contract is entered, it first checks to see if the flag is set, meaning that this call is reentrant. This allows the contract to revert if the reentrant call should not be allowed. Vyper provides a builtin @nonreentrant annotation for functions. In Solidity, this is typically implemented as a function modifier:

```
bool private _locked;

modifier nonReentrant() {
    require(!_locked, "Reentrant␣call");
    _locked = true;
    _;
    _locked = false;
}
```

Note that `_;` represents the body of the function the modifier is being applied to – that is, `_locked = true;` happens before the function body is executed, and `_locked = false;` afterwards. This type of modifier is also available in OpenZeppelin's widely-used smart contract library [29].

While reentrancy guards are more reliable than CEI in that they are enforced at runtime, they still have critical limitations. For instance, they must be manually applied by the developer. Given the non-trivial gas overhead induced by them, developers may selectively apply them only as they deem necessary, reintroducing the risk of oversight. Moreover, some projects explicitly require reentrant behaviour, in which case some reentrant calls must be permitted. This results in two distinct problems: First, any external call within the contract could reenter into the permitted entrypoint, regardless of what the developer's intent is. Even if one of the calls can safely reenter, any others may be in an inconsistent state. Regardless, the lack of a reentrancy guard would allow those external calls to reenter. The second problem has to do with what happens when the contract is first entered through an unguarded entrypoint. In this case, no flag is set at all – if an external call is now made, it will be able to reenter *any* entrypoint of the contract! This is even worse, since an attacker can now control which reentrant call is made.

3.2.3 Static Analysis

Several publicly available static analysis tools exist that are capable of detecting potential reentrancy bugs in Solidity code. Most notably, Slither [18] and Mythril [8] analyse the control flow and call graph of a contract, flagging cases where an external call precedes a state modification – a violation of the CEI pattern.

While this is useful, such tools have fundamental limitations in their application. First of all, any reentrancy guard will by definition be flagged by such analysis, since the lock flag is reset after any external calls. In general, such static analyzers often tend to report a vast amount of false positives [22], since they usually prioritize soundness over precision. Also, they are only useful if they are actually used – again, it is just an optional mechanism that relies on developer discipline. Finally, static analysis across call boundaries is fundamentally limited, since the behaviour of external contracts is unknown.

3.2.4 Comparison with Our Approach

In contrast to the above mechanisms, our language addresses reentrancy *at the language level*, making protection the default rather than an opt-in. Instead of guarding entrypoints, we introduce **path-specific** reentrancy annotations. By default, all reentrant calls are rejected. When a contract legitimately requires a reentrant call, the developer can opt in to exactly the required call path, nothing more. This is a significantly safer approach than leaving entire entrypoints unguarded, since it minimises the attack surface. Furthermore, since the protection is enforced by the compiler, it cannot be accidentally omitted. Lastly, the guarantees provided by the path-specificity enable analysis across reentrant call boundaries. This mechanism will be expanded upon in Chapter 4.

Chapter 4

Methodology

4.1 Core Ideas

The previous chapters have established that reentrancy is the most damaging type of vulnerability amenable to language-level solutions, that existing languages leave too much responsibility to the developer, and that code auditability is a critical but under-served property in this ecosystem. We identify three design principles that a language must satisfy to address these challenges: It should enforce *safety by default*, requiring developers to explicitly opt in to unsafe behaviour; it should make all side effects *explicit and visible*, so that auditors can reason about code without tracing through implementation details; and it should provide sufficient *expressivity* to natively model common patterns, shifting correctness guarantees to the compiler. The remainder of this chapter describes how each of these principles is realised in our language design, as well as the design decisions that led us to implement them as a new language rather than extending an existing one.

4.1.1 Reentrancy Protection

As we have seen, reentrancy vulnerabilities are one of the most prevalent, yet easily classifiable kinds of exploits. While the exact attack pattern may vary, it must by definition always involve a nested call into a contract that is already mid-execution. Interestingly, most smart contracts do not even require reentrant calls for their intended purpose. In fact, many contracts that have been exploited via reentrant calls did not require reentrant calls to work at all. In many cases, the issue was simply a lack of adequate protection against reentrant calls.

Given the high financial stakes in this environment, it seems clear that allowing reentrant calls by default is not ideal. Most projects do not use reentrancy at all, and thus developers should not be forced to reason about such cases unless they intend to specifically make use of reentrant calls. This does mean that the language no longer has a one-to-one correspondence with the behaviour of the EVM itself. However, this allows inexperienced developers to simply reason about their own code without needing to worry about the intricacies of the underlying virtual machine.

Existing approaches to reentrancy protection almost always involve adding specific permissions to the entrypoints of the contract. For example, standard practice in Solidity and Vyper is to add a `nonReentrant` modifier to all external functions that do not need to be reentered into. If the default behaviour were simply inverted, we would then expect the entrypoints to be labelled with `reentrant` modifiers where necessary. This is already a vast improvement – any project that does not require reentrancy at all would by definition not have to add any

such labels at all, while still being fully protected against reentrancy vulnerabilities. In fact, this would already have prevented 37 out of the 54 reentrancy exploits in our dataset.
However, there are still some issues regarding contracts that *do* require reentrant behaviour. A typical contract will not just make one external call, but make many different calls to various other contracts. It may well be the case that one of these external calls is expected to be reentrant, but that doesn't necessarily mean that it would be safe for *every* external call to then call back into the original contract. Yet with granularity at the level of entrypoints, there is no way to restrict this without the user implementing their own logic on top of the existing protections.
The other natural approach would be to decorate the call site with a `reentrant` label of some kind. Again, this leads to a similar issue - while a reentrant call to one entrypoint might be safe, any other entrypoint might lead to a bug. Thus, it seems clear that being able to map from a specific external call to a specific entrypoint is necessary.
When considering this from a "safety-by-default" perspective, this also makes sense. The developer should be able to allow exactly the behaviour they need, without allowing any further reentrant calls to be made.

Our solution

We propose the following approach: All external calls are not able to reenter the contract by default. The annotation of reentrant behaviour is made at the *external call* site, and must specify the *exact* entrypoint which is intended to be reentered.
For example, let's say we are making an external call to `address` with calldata `data`, and this call should be allowed to reenter the `repay(...)` function. This could be specified as follows:

```
EVM::reentrant_call(address, data, Self::repay)
```

Contrast that to a regular external call, which would be:

```
EVM::call(address, data)
```

The syntax restricts the reentrant call path to exactly what the developer would expect during normal execution. It does not impose much overhead – we simply add a reference to the corresponding entrypoint, which the developer surely has in mind anyway. Additionally, the control flow becomes much more obvious: If we consider an auditor reading through the code line-by-line, it is immediately obvious that the above call might lead to a call back to the `Self::repay` function. However, if only the entrypoint itself were annotated, then the transfer of control flow would not be clear.

4.1.2 Side Effects

One might say that the entire purpose of smart contracts is to produce side effects on a blockchain. Thus, considering side effects and their consequences is clearly a central part of reasoning about smart contract safety. This is especially relevant for smart contract auditors – given a codebase they have never seen before, they are tasked with ruling out bugs and vulnerabilities within the project, often within only a few weeks. Thus, it is vital for them to be able to reduce the search space as much as possible. We have already shown how this can be done by removing all non-necessary reentrant calls, but the same logic is also valid for side effects.
For illustration, consider something as simple as this:
`f(g(...), h(...))`

What can we say about the side effects produced by this expression? Clearly, `f` must be executed last, since it relies on the results of the other calls. However, the evaluation order of `g` and `h` is ambiguous. Of course, it depends on the language specification – but ideally an audit-oriented language would prevent ambiguous syntax in the first place.
In the specific cases of Solidity and Vyper, the situation is even worse: Not only is the syntax ambiguous, the evaluation order is actually *undefined* in such cases! That is, it is up to the compiler to choose whatever order it deems most convenient. While the evaluation order is almost always left-to-right in practice, there are some exceptions, as demonstrated by the winning submission to the **Underhanded Solidity Contest 2022** [20]. Of course, this specific behaviour is mostly limited to code that is designed to be misleading. However, such intentionally hidden bugs can be just as dangerous as unintended ones.
In this work, we have chosen to focus specifically on *state modifications*. They are likely the most relevant type of side effect for this application, but the principles used can be generalized to many other classes of side effects, such as logging (event emission), state reads, exceptions (reverting of transactions), and so on.

Our solution

Our guiding principles in this case were the following:

- There should be no hidden or implicit side effects.
- The ordering of effects must always be unambiguous.

Let's first look at an illustrating example.

```
contract Counter {
  n: u256
}
impl Counter {
  pub fn foo(self) {
    let val_1 = self.n;

    self.increment();

    let val_2 = self.n;
    assert(val_1 == val_2);
  }
  fn increment(self) {
    self.n += 1;
  }
}
```

Here, we can see that there is a "hidden" side effect when calling `self.increment()`. This may result in the incorrect assumption that `self.n` does not change during this call. While it may be obvious in such a simple example, when considering internal function calls in general, it's not so simple. In a sense, the side effects are "hidden" inside the implementation details of the function being called, so we cannot know the possible side effects without first fully understanding its implementation.
There are two issues at play here – let's first consider the hidden nature of the side effect. If we want to avoid such cases, the developer must somehow specify that `self.n` may change when `self.increment()` is called. There are many potential ways of doing this, but for now let's simply surround the relevant statement with a *permission block*:

```
    modify(self.n) {
      self.increment();
    }
```

Note that we would not need this decoration for *direct* side effects, such as the `self.n += 1;` statement later on. This is because the side effect is visible in the syntax, it is not hidden. This still leaves us with the second issue – when calling a function, we may not know all potential side effects that might be induced. The caller should not be required to understand the precise implementation details of a function. All relevant properties of a function should be provided by its signature. We can amend this by requiring the `increment` function to declare that it might modify `self.n`. That way, we encapsulate the information about potential side effects behind the signature of the function.

```
  fn increment(self) modifies(self.n) {
    self.n += 1;
  }
```

In this case, we have used a different keyword for the function-level declarations as opposed to the block-level permissions. This is mostly to make clear that they do not mean the same thing – `modify` is used to annotate *indirect* side effects occurring within the body of a function, while `modifies` is an element of the function signature, declaring the possible side effects that may occur when the function is executed. Note that it is likely that this choice of syntax would be confusing to new developers. However, the main purpose here is to demonstrate this as a proof of concept, the specific syntax could always be changed.

4.1.3 Side Effects in External Calls

There is one obvious limitation to dealing with side effects in this way – what happens if we don't know the details of the function being called? This is an unavoidable situation when a contract makes an external call into untrusted code. As soon as the control flow transfers to arbitrary code, the compiler is unable to enforce its own rules regarding side effects. This makes it impossible to enforce properties about *external* state. In fact, it is even difficult to reason about *internal* state at all, due to the possibility of reentrant calls. This is where our first core feature comes to the rescue.

```
contract Counter {
  n: u256
}
impl Counter {
  pub fn foo(self) modifies(self.n) {
    let addr: Address = EVM::caller();
    let data: Bytes = Bytes::empty();
    modify(self.n) {
      EVM::reentrant_call(addr, data, Self::increment);
    }
  }
  pub fn increment(self) modifies(self.n) {
    self.n += 1;
  }
}
```

As it turns out, we have all the information we need – every external call site must specifically declare which functions it may reenter. As we saw before, every function must now declare its possible side effects. These two facts let us guarantee which internal side effects may happen during any external call *at compile time*. This type of guarantee would not otherwise be possible without the use of sophisticated static analysis to rule out reentrancies.
Thus, it becomes even simpler to reason about reentrancy vulnerabilities. Not only do we know exactly what entrypoint is being called, it is also immediately clear at the call site which state variables might be modified in the nested call, despite the external call executing arbitrary code. In fact, if the side effect declarations were extended to also include state reads, we could also much more easily rule out read-only reentrancy bugs as well.

4.1.4 Reentrancy as a Side Effect

There is a further kind of hidden effect we have not yet addressed. Consider the following example:

```
contract Counter {
  n: u256
}
impl Counter {
  pub fn foo(self) {
    let val = self.n;
    self.bar();
    self.n += 1;
  }
  pub fn bar(self) {
    EVM::reentrant_call(addr, data, Self::foo);
  }
}
```

As previously mentioned, we want to avoid hidden side effects in the code. There is another hidden effect at play here – from the perspective of the `foo` function, it is not clear that it may be called in a reentrant context. This is an especially dangerous situation, since by definition there will be a possible overlap between the effects produced by the reentrant call and those in the top-level call. Even if the reentrant call were made to some other entrypoint, it is still vital to carefully consider what exactly may happen during the reentrant call. Therefore, we treat reentrant calls the same way as the state modifications – every function signature must declare its possible reentrancies, and any indirect reentrancies within a statement must be surrounded by a permission block.

```
  pub fn foo(self) reenters(Self::foo) {
    let val = self.n;
    reenter(Self::foo) { self.bar(); }
    self.n += 1;
  }
  pub fn bar(self) reenters(Self::foo) {
    EVM::reentrant_call(addr, data, Self::foo);
  }
```

The overhead induced by this requirement is quite small compared to the state modifications, since reentrant calls are much less common. But we will discuss the overhead of the effect

annotations in more detail later on.
There is one edge case we have neglected to address so far. How can we deal with reentrancies that make further reentrant calls? What about a recursive function reentrantly calling itself? In general, this is an exceedingly rare occurrence. In fact, it seems unlikely for any use case to require nested reentrancies at all. Thus, it would be reasonable to simply disallow this case from occurring at all. Even so, there is no real barrier to reasoning about nested reentrancies with the same logic we have used so far. The caller of such a function must simply declare all the possible further reentrancies, which may include the calling function itself.

4.1.5 Algebraic Data Types

Algebraic data types are included primarily for their expressivity – they allow developers to natively model complex state machines. Thus, we reduce the need for manual re-implementations that are more prone to bugs. Another compelling use case is provided by the `Option` and `Result` types, which natively handle nullable values and potentially failing computations respectively. This is a significant improvement over the "silent failure" in Solidity, where a failed external call may return `false` instead of reverting – meaning a developer must check the return value manually. Further, the exhaustiveness checks enforced by the compiler ensure that all possible cases must be covered, leaving no room for developer error.

4.2 Discussion

Having established the three core features, we now consider how best to make them available to developers. The most obvious solution would be to modify an existing widely-used language such as Solidity or Vyper. However, this would present significant challenges, as we will see in a moment. Most other projects targeting EVM are significantly out of date, and often have their own unique feature sets which do not compose well with ours. Additionally, there is little benefit in modifying a language that is not already being used for smart contract development, since the language will have been designed with other goals in mind.
Unfortunately, that seems to indicate the necessity of designing a new language which specifically fits our desired features. In the interest of making it easy to adopt, it should be based closely on an existing well-known general-purpose programming language. As the preceding code examples suggest, the language chosen in this case is Rust.

4.2.1 Retrofitting Solidity

Solidity is the oldest, most widely used programming language for EVM. As such, there is also a large incentive not to break backwards compatibility. While there have been breaking changes in the past, they are quite rare. In fact, since the release of Solidity v0.8 in December 2020, no more breaking changes have been made at all.

ADTs

Adapting Solidity to allow ADTs would not present too much of a challenge. Assuming they were handled similarly to Rust, one could simply extend the concept of a Solidity

`enum` to include fields. Although one might have to reconsider the syntax of a `switch` statement, or alternatively add some new syntax to enable variable bindings when pattern matching. Of course, great care needs to be taken to ensure backwards compatibility not only in the syntax, but also in the ABI encoding of types and function calls, since newly deployed contracts should still be able to interact with all existing contracts.

Reentrant Call Paths

As long as we only require reentrant external calls to declare their corresponding entrypoints, this is again just a small syntactic change to the language. However, the `reenters(...)` declaration in function signatures does not play well with Solidity's contract inheritance and virtual functions. This is not necessarily a deal-breaker, but does present some real obstacles. For example, how could one define an overridable function in an abstract contract, such that it is allowed to reenter a function in an implementing contract? There are solutions, but they are all work-arounds of some kind. Additionally, one would have to be able to differentiate between overloaded functions, and between the functions defined on the current contract versus those on its super-types (i.e. `reenters(super.foo)` vs. `foo`).
What's more, Solidity also features function pointers. One would either have to prevent creating pointers to reentrant functions entirely, or extend the function pointer type to also include potential reentrancies. However, it would not be generalizable – if we wanted to pass `function() reenters(this.foo)` and `function() reenters(this.bar)` as arguments to the same function, we would need to be able to generalize over the function to be re-entered. Solidity has no notion of generics or type variables, so this would not be a natural extension to the language.

Effects

Similar to the reentrancy annotations, the main obstacles here are provided by contract inheritance, as well as function pointers. The difficult questions that would need to be answered are:

- How can an abstract contract specify a modification to a state variable of the implementing contract?
- How can we extend the notion of function pointers to naturally include their effects?

Again, we believe these to be prohibitively large obstacles to extending Solidity with our proposed features.

4.2.2 Retrofitting Vyper

The design of the Vyper language is much simpler by comparison. It is closely based on Python, but intentionally restricts its feature set so as to remain as simple as reasonably possible. Due to its widespread use, the language is similarly tied up with the responsibility of maintaining backwards compatibility, although less so than Solidity. Thanks to the simplicity of the language, there are fewer obstacles to implementing our proposed features in the language. However, the philosophy behind Vyper (and Python) is more about making the language easy to use, and reducing developer overhead. Furthermore, Vyper restricts access to low-level EVM features such as memory allocations, which makes it tricky to reduce the memory footprint. For that reason, it seems unlikely that such features would ever be adopted by the Vyper compiler team. Even so, we will briefly discuss the viability of each feature in detail.

ADTs

There is no real barrier to implementing ADTs and pattern matching in Vyper, since Python already has support for such types. One slight issue is that Vyper does have reference types, but when passing these types to functions they are implicitly copied. This incentivizes less abstraction and shallower call hierarchies. As such, top-level code that involves many ADTs would be littered with pattern matching case distinctions, rather than "hiding" such functionality within methods defined on the type itself. This makes the feature much less attractive to use, especially since many of Vyper's advantages are provided by its simplicity and straightforward code. Furthermore, Vyper does not give the developer precise control over when memory allocations happen. Due to the frequent copying of memory, as well as the larger memory footprint induced by ADTs, it would likely be tricky to keep allocations below a reasonable limit.

Reentrant Call Paths

Although Vyper does not prevent reentrancies by default, it is likely for this to be changed in the near future. However, due to the desire for simplicity, this will be handled via a `@reentrant` modifier on the entrypoints. For this reason, it again seems like the restrictive nature of our feature set is not a good fit for Vyper, even though there are not necessarily significant barriers preventing such an implementation.

Effects

Vyper provides modularity through composition. This avoids the problematic function overriding and overloading we encountered with Solidity. The language also does not feature function pointers, which was the other major road-block. Thus, the annotation of side effects would actually fit quite well into the language design, especially since it already has annotations for function declarations. However, *requiring* the side effect annotations to be provided by the developer would go against the language's philosophy, and present a huge breaking change to all existing code. Thus, it would be exceedingly unlikely for this kind of change to be adopted or well-received by existing users of the language.

4.2.3 Rust as a Base Language

As demonstrated by Vyper, it can be advantageous to design a niche language closely on an existing general purpose language, so as to ease adoption for new developers. Initially, we had planned to design a functional language, likely syntactically similar to Haskell. However, the decision was made early on to instead make the language imperative, since functional languages tend to be more difficult to learn and are not widely used. Rust was the next natural choice, since it features ADTs and pattern matching natively. In fact, there are not many mainstream imperative languages that do support pattern matching. The somewhat low-level nature of Rust also is a benefit, since we do want to give the developer control over what operations are happening, most notably memory allocations and storage reads / writes. It is of course still possible to build up many layers of abstraction, for example by defining methods on user-defined types or using traits. Beyond the explicit features, the Rust-inspired design also naturally encourages a separation between pure and effectful code, since methods can be defined on user-defined types directly, rather than within contract space.

Chapter 5

Implementation

Having established the design principles and their theoretical motivation, this chapter describes how they are realised in a working compiler implementation. The implementation of our compiler consists of roughly 30'000 lines of Rust code, as well as another ~10'000 lines of code for tests. The compiler pipeline is of relatively standard design, with the novel addition of a pass to check the effect annotations. The following sections describe each stage of the pipeline in detail. All in all, it consists of 13 passes, although some similar functionalities are split up into multiple passes simply because not all the necessary information is available the first time around. For example, syntax desugaring is applied once before type inference, and again afterwards – this is an unfortunate consequence of the type system, since it is not clear right away whether an expression such as `Type::Member` refers to a constant, function, or enum variant (or an associated type, which we do not support). Moreover, the member may be present in several different traits or `impl` blocks, so until type inference occurs we can't necessarily deduce which one it refers to. Each step in the pipeline is explained in more detail in the following sections.

5.1 Pipeline

The pipeline consists of the following stages:

- Lexing
- Parsing
- Desugaring
- Linking
- Type Inference
- Effect Checks
- Monomorphisation
- Code Generation

5.2 Lexing, Parsing and Desugaring

The source code is initially converted into a stream of tokens. These tokens consist of symbols, keywords, identifiers and literals. Note that there is also a slight distinction

between keywords and identifiers with special meaning. For example, `return` is a keyword, which can never be used to mean anything other than a `return` expression. However, `self` is a special identifier which can only be used in certain contexts, but does otherwise behave as any other identifier would.

The parsing is responsible for converting this token stream into the actual abstract syntax tree representation, which will be used for the remainder of the compilation pipeline. Most parsing functionality is implemented through parser combinators, although expression parsing is manually programmed to follow certain precedence rules, and to avoid ambiguities. For example, a struct constructor cannot be used as a top-level expression in a match scrutinee, since this would lead to ambiguous parsing:

```
match Struct { ... } { ... }
```

This could be a match expression scrutinizing a struct literal, or it could be a match on an expression consisting of the identifier `Struct`, followed by the match body and then a regular code block. This second choice would likely fail to parse, but it might take an arbitrarily large look-ahead to figure this out, so it is simpler to just disallow struct constructors in this position.

The syntax desugaring helps simplify the compilation process by reducing the number of syntax elements that can occur. This is done by rewriting certain constructs into others. For example, any `if` statement can be represented through an equivalent `match` expression. In fact, every variable declaration is actually equivalent to a match expression with a single arm! This avoids having to do similar pattern matching checks for destructuring variable declarations, such as `let (x, y) = e;`.

Some built-in operators are also actually defined as traits, so that `a + b` is converted into a trait method call such as `Add::add(a, b)`. This conversion is also done at this point.

One aspect of desugaring that is done later on is specifying path identifiers to be enum variants. This avoids having to look up the resolution of an identifier to distinguish between variables and enum variants, for example in pattern matching. The issue stems from the fact that `Option::Some(x)` could also be a function call, or `Option::None` could actually be referring to a constant. But this can only be determined once more information is available.

5.3 Linking

The linking is executed in two phases. First, all non-local definitions are collected and stored according to their module hierarchy. Then, we can pass through the AST a second time, resolving each reference to its appropriate definition. It is not always possible to uniquely resolve every reference at this stage, since we are still lacking type information, as previously mentioned. Therefore, any expressions involving trait members or similar are resolved as far as they can be, then stored for later inference.

References to local variables are much simpler to resolve, since we can simply visit the AST in order and simply keep track of which variables are in scope.

5.4 Type Inference

The type inference is likely the most complex element of the compiler. The issue is a typical chicken-and-egg situation – not all references can be resolved prior to type inference,

but the types of those references cannot be known until they are resolved. Furthermore, soundly dealing with generics and trait bounds is a tricky business, since such constraints must be propagated and subsequently checked to be valid. We implement a version of Hindley-Milner type inference, loosely according to the approach described in *Typing Haskell in Haskell* [25], an excellent paper describing how a relatively similar type system can be soundly inferred. The main difference is that Haskell's class definitions differ slightly from Rust's traits, in that they do not take type arguments. That is, the type implementing a Haskell class may take type arguments (such as with Haskell's Monad class), but the class itself does not directly constrain the argument type. By contrast, Rust traits explicitly take type arguments, and may even constrain them to fulfill certain trait bounds. This mainly changes how one must detect and deal with overlapping trait instances.

5.4.1 Notation

We first define a few notational conventions.

Symbolic Types

Since we are dealing with parametric polymorphism (generics), we also necessarily introduce the concept of symbolic types. Moreover, we need to be able to express trait bounds. The notation `<T: Foo> T` essentially means "some type T that implements the trait `Foo`".

Sub-Typing

We define the sub-typing relation between types, $T_1 <: T_2$, meaning that T_1 is a sub-type of T_2. Note however, that this sub-typing relationship differs slightly from what you might be used to from object-oriented languages. Rust has no inheritance, instead what is meant by this is that a type is "more specific" than another. For example, the type `<T: Integer> T`, meaning some symbolic type T that implements the trait `Integer`, is "less specific" than the type `u256`. Intuitively, this is because T could refer to any type that is an integer, whereas `u256` refers to a specific integral type. In general, $T_1 <: T_2$ holds if there is some set of substitutions σ to the bound type variables in T_2 such that $\sigma(T_2) = T_1$.

Unification and Matching

There are two important operations on types for our type inference. The first is the *most general unifier* (*mgu*). Given two types A and B, $mgu(A, B)$ is defined to be the "most general" type T such that $A <: T$ and $B <: T$. "Most general" means that for *any* type T' with $A <: T$ and $B <: T$, then $T' <: T$ must also hold. Therefore, T is at least as general as any other type unifying A and B, thus making it the most general unifying type.

The second operation is matching one type to another. For it to yield a valid result, $B <: A$ must hold. The goal is to find a set of substitutions σ, such that $\sigma(A) = B$. For example, matching `<T: Integer> T` to `u256` would give the substitution $T \rightarrow$ `u256`. This operation is useful when assigning one value to another, since the left-hand-side of an assignment must have a more specific type than the right-hand-side. For example, the following assignment *is* valid:

```
let x: u256 = Default::default();
```

Whereas this one is not:

```
let x: T = 0u256
```

In the first example, the right-hand-side is simply some symbolic type `<D: Default> D`, and thus can be matched to `u256`. In the second example, while T is symbolic, it could refer to any type and may not be constrained.

5.4.2 Inference

The general approach to our type inference is to assign each expression a fresh symbolic type. Then, whenever two symbolic types are combined to form a compound expression, they can be unified according to the constraints induced by this expression. For example, let's define two variables `a` and `b` with symbolic types A and B. To infer the value of the expression `a + b`, the syntax must first be desugared. The types must implement the `Add` trait, therefore we can safely rewrite the expression to `Add::add(a, b)`. Note that `a.add(b)` is *not* a safe rewrite, since the type A may also have other methods with the same name. Now, we can check the trait's definition to see what constraints must apply to A and B for this to be valid. Let's say the definition of the method is as follows:

```
fn add(self, other: Self) -> Self
```

We can infer that A must equal B, as well as that A must implement the `Add` trait. Thus, we infer the type of `a + b` to be `<A: Add> A`. This trait bound must be propagated until A can be concretized, or until it is unified with a generic type with the corresponding trait bound.

Alternatively, there may be more indirect ways of satisfying the bound. For example, if we find that A implements trait `Foo`, but that `Foo` lists `Add` as one of its super-traits, then the bound can be dropped as well. Alternatively, there may be some instance of the `Add` trait that applies to A, even if the type is not yet concrete. This is often the case for types such as `Option<T>`, since we can define addition over the `Option` type by requiring `T: Add`.

Recall the important distinction between *unifying* types and *matching* them. This becomes especially critical when dealing with variable assignments and inferred types. In an assignment such as `foo.x = y;`, we cannot safely modify the type of the left-hand-side, even if it is not a concrete type. The main exception is if we have a novel variable declaration with an inferred type, such as `let x: _ = y;`. Here, we assign `x` a new symbolic type, but keep track of the fact that it stems from a type that should be inferred, meaning it can safely be unified with RHS types.

5.4.3 Method Resolutions

The trickiest part of the type inference is to correctly resolve invoked methods to their definitions. As previously mentioned, the resolution requires type information (e.g. to know if a type implements a certain trait), but the type inference requires knowing which method is being referenced. Therefore, we cannot necessarily reject ambiguous method references right away. Instead, we run through the entire type inference once, inferring as many types as possible. If there are still unresolved symbolic types, we revisit them to see if the methods can now be uniquely resolved. If there is still an ambiguity at this point, we reject the program as being invalid (although it may still have a uniquely correct type inference, but this should be exceedingly rare).

5.5 Effect Checks

The effect tracking system is by contrast much simpler than type inference. Certain operations are defined to trigger *direct* side effects, such as writing to a storage field, or calling `EVM::reentrant_call`. These effects are aggregated along with any other inline effect declarations, in order to determine which side effects the containing function must declare. Here, we also allow declaring in a less precise way, for example `modifies(self.foo)` would encompass both `modifies(self.foo[0])` and `modifies(self.foo[1])`. Side effects can also be declared in relation to function arguments, so an effect declaration such as `modifies(self.foo[index]` is valid if `index` is an argument to the function (or a global constant). For reentrancy annotations, it is simply required to declare all entrypoints that may be reentered within the function body. It may be useful in the future to create shorthand syntax to allow reentering all read-only functions, without having to specify them all. But this again risks developer oversight since not every entrypoint is being carefully considered in isolation.

It is worth noting that it may be difficult to precisely annotate certain side effects, for example modifying an arbitrarily long range of elements within an array. For instance, consider the following function:

```
fn clear(self, from: u256, to: u256) {
  for i in from..to {
    self.array[i] = 0;
  }
}
```

It is not possible to precisely annotate these effects – while we could introduce Rust's range syntax, it is not difficult to construct an example where the written elements are not contiguous. Without introducing some complex notion of preconditions / postconditions, we simply have to accept that the annotations will not be perfectly precise. In the above case, we can simply annotate `modifies(self.array)` to encompass all the possible side effects.

Another interesting case is well-illustrated by an example of a real-world exploit, namely that of *Super Sushi Samurai*. Their contract contained a transfer function, which contained the following (slightly simplified) logic:

```
function _update(address from, address to, uint256 amount)
  internal override virtual
{
    // don't check if it is minting or burning
    if (from == address(0) || to == address(0) || to == address(0xdead)) {
        super._update(from, to, amount);
        return;
    }

    uint256 fromBalanceBeforeTransfer = _balances[from];

    uint256 amountAfterTax = amount - _taxApply(from, to, amount);
    uint256 toBalance = _balances[to] + amountAfterTax;

    _balances[from] = fromBalanceBeforeTransfer - amount;
    _balances[to] = toBalance;
```

```

    _unlockTokenForDev(from, to, amount);

    emit Transfer(from, to, amountAfterTax);
}
```

The relevant issue for us is that there is no check to ensure that the `from` address is different from the `to` address. Ignoring the tax applied to the transfer, we can see that `_balances[from]` is reduced before `_balances[to]` is set to `old(_balances[to]) + amountAfterTax`, where `old(x)` refers to the value of `x` before the function execution. This results in the incorrect logic when `from == to`, allowing a user to simply transfer funds to themselves, duplicating the sent amount each time.
The interesting part for us is to determine how the compiler should handle this situation, given the right annotations are in place. For example, observe the following function:

```
fn transfer(self, from: Address, to: Address, amount: u256)
  modifies(self.balances[from], self.balances[to])
{
  let from_balance = self.balances[from];
  let to_balance = self.balances[to];
  self.balances[from] = from_balance - amount;
  self.balances[to] = to_balance + amount;
}
```

It is not exactly clear how the compiler should deal with this – of course, we could just ignore the potential aliasing issue and compile it normally. However, it is not too far-fetched to expect the compiler to at least warn the user about this. The difficulty is simply to be able to emit the warnings precisely, i.e. not give too many false positives. The necessary logical conclusions that would have to be made here are the following two:

- `from` is not checked to be different than `to`
- If `from == to`, the write to `self.balances[to]` would directly overwrite the write to `self.balances[from]`, without that value ever being read.

However, we would ideally not like the compiler to have to reason about the code itself, i.e. whether or not `from == to` is satisfiable, since that may become an arbitrarily difficult problem to solve. Thus, this is currently a situation that our compiler cannot warn the user about. What it *can* do, however, is to inform the user when two such side effects are ambiguously ordered. That is, if two potentially overlapping side effects occur in the same line of code, in such a way that both possible orderings are reasonable outcomes, then the program will be rejected. Needing to enforce this is a consequence of the undefined evaluation order in Solidity, which is the target representation that our compiler produces. Therefore, to ensure safety of the produced side effects, we must reject ambiguities.

5.6 Monomorphisation

The monomorphisation stage concerns itself with projecting code that deals with generics onto concrete representations. Essentially, we need to gather all possible instantiations of generic elements, then create one copy of that element for each instantiation. More concretely, if we have an enum such as `Option<T>`, and it is used in several different

contexts (e.g. `Option<u256>`, `Option<Address>`, `Option<Option<()>>`), then we want to copy the code once for each unique instantiation of this type.

Unfortunately, it's not quite as simple as just seeing what all the concrete type instantiations are. Since generic code may then call into further generic code, we have to transitively apply whatever substitutions are necessary. To illustrate, let's explore a non-trivial example:

```
enum Option<T> {
  Some(T),
  None
}
trait Add<T> {
    fn add(self, t: T) -> Self;
}
impl<T2, T1: Add<T2>> Add<Option<T2>> for Option<T1> {
    fn add(self, t: Option<T2>) -> Option<T1> {
        match (self, t) {
            (Option::Some(a), Option::Some(b)) => Option::Some(a + b),
            _ => Option::None,
        }
    }
}
impl Add<bool> for u256 {
    fn add(self, t: bool) -> u256 {
        self
    }
}
impl Add<u256> for bool {
    fn add(self, t: u256) -> bool {
        self
    }
}
fn foo() {
    let x: u256 = 0u256;
    let y: bool = true;
    let z1: bool = Add::add(y, x);
    let z2: u256 = Add::add(x, y);

    let ox: Option<u256> = Option::Some(x);
    let oy: Option<bool> = Option::Some(y);
    let oz1: Option<bool> = Add::add(oy, ox);
    let oz2: Option<u256> = Add::add(ox, oy);
}
```

What makes this code difficult to compile, is the fact that we want to abstract addition over the `Option<T>` type, while only requiring that T itself implements the `Add` trait, but without knowing exactly which type it refers to. First, we will gather all the references between definitions (and their type parameters). In the above example, that would be as follows:

- `enum Option<T>`: $\varnothing$

- `trait Add<T>` : $\varnothing$
- `impl<T2, T1: Add<T2>> Add<Option<T2>> for Option<T1>` :
  - $\rightarrow$ `impl Add<T2> for T1`
  - $\rightarrow$ `trait Add<Option<T2>>`
  - $\rightarrow$ `enum Option<T1>`
  - $\rightarrow$ `enum Option<T2>`
- `impl Add<bool> for u256` :
  - $\rightarrow$ `trait Add<bool>`
- `impl Add<u256> for bool` :
  - $\rightarrow$ `trait Add<u256>`
- `fn foo` :
  - $\rightarrow$ `enum Option<u256>`
  - $\rightarrow$ `enum Option<bool>`
  - $\rightarrow$ `impl Add<Option<u256>> for Option<bool>`
  - $\rightarrow$ `impl Add<Option<bool>> for Option<u256>`

Now, we can determine what our top-level entrypoints are. In normal situations, this would simply be a list of all external contract entrypoints. Here, we will just assume `fn foo` is the only top-level entrypoint. Thus, we start with that as our only instantiation:
{ `fn foo` }
Now, we can look up all the references made by this function, and add them to our list.
{ `foo` , `enum Option<u256>` , `enum Option<bool>` , `impl Add<Option<u256>> for Option<bool>` , `impl Add<Option<bool>> for Option<u256>` }
Now, we continue by processing the next element in the list. Since `Option` has no further references, we can skip the next two.
The first interesting case to look at is `impl Add<Option<u256>> for Option<bool>` . Since the trait implementation takes type arguments, we infer the following substitutions: $\sigma = [\texttt{Self} \rightarrow \texttt{Option<T1>}, T1 \rightarrow \texttt{bool}, T2 \rightarrow \texttt{u256}]$
Now, we apply these substitutions to any further references made by this definition:

- $\sigma($ `impl Add<T2> for T1` $) =$ `impl Add<u256> for bool`
- $\sigma($ `trait Add<Option<T2>>` $) =$ `trait Add<Option<u256>>`
- $\sigma($ `enum Option<T1>` $) =$ `enum Option<bool>`
- $\sigma($ `enum Option<T2>` $) =$ `enum Option<u256>`

This is how we are able to infer that the implementations for adding `u256` and `bool` to each other must be preserved. Each new instantiation we create must then also be recursively processed in this manner, until we have traversed the entire list. Now, we can create copies of the instantiated definitions, by duplicating the code while applying the appropriate substitutions. For example, the instantiation `enum Option<bool>` becomes:

```
enum Option_bool {
  Some(bool),
  None
}
```

This also means that we must replace every use of `Option<bool>` with the new `Option_bool`.

Of course, any code that did not deal with generics in the first place need not be duplicated, and we will only create instantiations of definitions that are actually made use of directly or indirectly in external entrypoints. Thus, this stage also serves as a form of dead code elimination, since we can discard any definitions that are never used. Finally, we can also discard all definitions that use generics, since we have replaced all references to them with concrete implementations.

5.7 Code Generation

The translation of Rust code into Solidity code is actually not that complicated. Most structures can be roughly mapped one-to-one, at most with some minor tweaks. The major exceptions to this are tuple types, ADTs and `match` expressions.

5.7.1 Tuple types

In Solidity, a single local variable can never contain a tuple type, even though such types can be returned by functions. Instead, one is forced to destructure such assignments. We avoid this difficulty by defining Solidity structs to serve as tuples, through a very similar process to the monomorphisation stage just discussed. Essentially, anytime a tuple is used in Rust code, we replace it by a tuple struct with the appropriate elements. Note that we do have to define a struct for each unique combination of types. Furthermore, the tuple struct will be stored in memory, rather than on the stack as it would be with native Solidity tuples.

5.7.2 ADTs

The second major challenge is provided by ADTs. Since Solidity cannot natively represent these, we must come up with a workaround. There are several ways of doing this, but for now we have chosen the easiest - an enum to define which case of the ADT is in effect, as well as an accompanying struct that contains all fields of all enum variants. Clearly, this is very memory-inefficient. Unfortunately, it is not easy to generate more optimized code without generating inline assembly to take control over memory allocations. Since this compiler is still a proof-of-concept, we decided to prioritize a correct implementation over an efficient, but more error-prone one. To illustrate the approach, we include an example compilation of a simple ADT:

```
enum MaybeOneOrTwo {
  Two(u256, Address),
  One(u256),
  Zero
}
```

```
enum MaybeOneOrTwo_tag {
  Two,
  One,
  Zero,
}
struct MaybeOneOrTwo {
  MaybeOneOrTwo_tag tag;
  u256 Two_0;
  Address Two_1;
  u256 One_0;
}
```

5.7.3 Pattern matching

The final slight difficulty is compiling `match` expressions. These can be destructured into a series of `if` / `else if` statements. In principle this is not too difficult, the appropriate conditions to check may just be nested arbitrarily deep within the scrutinee of the match expression. Furthermore, we need to translate any bound variables within each match arm to assignments within the body of the corresponding `if` statement. Lastly, we need to make use of our representation of ADTs in Solidity to correctly check which case of the ADT is currently in effect. The rough approach can be seen in the following example compilation:

```
pub fn asdf(val: MaybeOneOrTwo) -> u256 {
  match val {
    MaybeOneOrTwo::Two(_, Address::ZERO) => {
      EVM::revert("zero address!")
    },
    MaybeOneOrTwo::Two(amt, addr) => {
      let bal = EVM::balance(addr);
      amt + bal
    },
    MaybeOneOrTwo::One(x) => x,
    MaybeOneOrTwo::Zero => 0,
  }
}
```

```
function asdf(MaybeOneOrTwo calldata val)
  external
  returns (uint256)
{
  MaybeOneOrTwo memory tmp = val;
  if (
    tmp.tag == MaybeOneOrTwo_tag.Two
    && tmp.Two_1 == address(0)
  ) {
    revert(string("zero address!"));
  } else if (tmp.tag == MaybeOneOrTwo_tag.Two) {
    uint256 amt = tmp.Two_0;
    address addr = tmp.Two_1;
    uint256 bal = address(addr).balance;
    return (amt + bal);
  } else if (tmp.tag == MaybeOneOrTwo_tag.One) {
    uint256 x = tmp.One_0;
    return x;
  } else if (tmp.tag == MaybeOneOrTwo_tag.Zero) {
    return uint256(0);
  }
}
```

Chapter 6

Evaluation

6.1 Reentrancy Protection

Clearly, the main goal of providing protection mechanisms to guard against reentrancies is to prevent real-world vulnerabilities. Thus, it is vital to consider how our approach would have impacted historical cases where reentrancy vulnerabilities led to an exploitable security flaw. To analyze this, we can use the vulnerability dataset introduced in the background chapter – it contains 54 cases of reentrancy exploits, totalling $187 million in lost funds. These cases can be further broken up into a few categories. Firstly, the majority of exploited contracts did not make use of reentrant behaviour at all, instead they simply did not adequately protect against it. For these 37 cases, simply rejecting all reentrant calls by default would have sufficed.

Next, there were 6 cases of read-only reentrancies – here, the contracts being exploited were not being reentered, instead they relied on a price oracle to determine a conversion ratio between two currencies. However, the oracle contracts did not protect their read-only functions against reentrant calls. Thus, the price could be manipulated for the duration of the reentrant call, and when the oracle price is read from, an incorrect value is returned. This results in the unfortunate situation where the contract being exploited is not really implemented incorrectly, instead it simply relied on an unreliable oracle. Because this type of exploit was first discovered in 2022, it is only recently that it has become standard to protect read-only functions against reentrant calls. In fact, applying the standard `@nonReentrant` modifier to a view function in Solidity is not possible, since the modifier itself writes to storage. Similarly, it was originally not possible to use the built-in @nonreentrant modifier on view functions in Vyper. Thus, we argue that these vulnerabilities were the result of a lack of appropriate tooling, as well as the knowledge that this type of vulnerability is even possible. Once again, this issue would have been avoided if the oracle contracts had simply rejected all reentrant calls.

The next category involves certain token standards. There are various token standards, such as ERC-777, ERC-677, and more, which mandate a `beforeTokenTransfer` (or similar) call to the receiver of a transfer, so as to give the receiver a chance to reject the transfer. However, as these standards are extensions to the ERC-20 token standard, they can be used in place of any other ERC-20 token. Thus, any protocol that involves ERC-20 tokens must carefully consider the possibility that a token they interact with might lead to reentrant calls. Another 6 of the reentrancy vulnerabilities were due to this oversight. While it is true that rejecting all reentrant calls might suffice in this case, it does also mean that interacting with some of the mentioned token standards would no longer be possible, due to the mandated

callbacks. Since the creators of these protocols did intend to interact with such callback tokens, it is by definition part of the use case they were intending. Thus, we argue that a more sophisticated mechanism would have been necessary to prevent these vulnerabilities.

Two of the vulnerabilities involved contracts with flash loan functionality. The usual way this works is the following: The external user calls the flash loan contract, specifying an address to a contract they control, as well as an amount of some currency to borrow. Next, the flash loan contract transfers this amount to the specified contract, then makes an external call to it. When this external call terminates, it asserts that the full balance has been repaid, along with some fee on top. Sometimes, the user is expected to make a reentrant call to a `repay` function on the flash loan contract, although this is not strictly necessary. In any case, it becomes extraordinarily important to handle reentrant calls correctly - if borrowing some funds sets the user's debt to the amount borrowed, then reentrantly borrowing a second time would overwrite this value. Furthermore, correctly tracking the balances to assert that the full debt has been repaid is again critical. Flash loans in general are extremely prone to being drained of funds, since the use case itself inherently involves transferring a large amount of funds to an address the user controls. For example, in the specific case of the DFX exploit, the same contract could also be used to simply deposit funds and withdraw them. However, the flash loan functionality simply checked the balance of the contract as a whole to see if the borrowed funds had been returned. Moreover, the `flash` function was the sole entrypoint without a reentrancy guard. Thus, the user could borrow the funds without triggering a reentrancy guard, then reentrantly deposit the funds regularly. The contract then saw that the funds had been returned, and erroneously assumed they had simply been transferred. Finally, the user could then withdraw to completely drain the contract of funds.

A further two vulnerabilities involved manually written reentrancy guards that intended to allow some reentrant functionalities but disallow others. Clearly, this use case required reentrant calls, but the adequate tools were not available to do this correctly. Hence, we again argue the importance of being able to granularly specify reentrant call paths.

The final case involved a bug in the Vyper compiler. The built-in reentrancy guards were not implemented correctly in some versions, allowing them to be bypassed. While it is true that rejecting reentrant calls would have prevented this, it is unfair to argue that the issue could have been addressed by language design. The developers wrote secure code, it was simply compiled incorrectly. Hence, we do not claim that our tools would have prevented this exploit.

These findings are summarized in table 6.1, where "Safety by Default" refers to whether or not the vulnerability would have been prevented by simply rejecting all reentrant calls, and "Path-Specific" refers to whether our proposed protection mechanism of specifying the reentrant call path would have sufficed.

6.1.1 Performance Overhead

Of course, no runtime safety checks come without a tradeoff. In this case, it's even quite costly – storage operations are one of the most expensive features of the EVM. However, offloading the responsibility of reentrancy guards to the compiler gives us an advantage: Instead of simply adding the same @nonReentrant modifier to every entrypoint, we can choose the cheapest approach at compile time. For example, a function that makes no external calls does not need to *write* to the guard variable, it simply needs to *read* it. The main downside we have compared to standard reentrancy protection is read-only functions.

Category	Count	Funds Lost	Safety by Default	Path-Specific (Ours)
Missing Guard	37	$151 million	Yes	
Read-only Reentrancy	6	$9.6 million	if oracle contract uses mechanism	
Callback Token	6	$57.6 million	incompatible	Yes
Manual Guards	2	$27.7 million	No	Yes
Flash Loan	2	$5.7 million	No	Yes
Compiler Bug	1	$41 million	Unclear	
Total	54	$187 million	43/54	53/54

Table 6.1: Preventability of 54 real-world reentrancy vulnerabilities (2021-2024)

In most cases, they are not guarded, meaning the guard variable is never accessed at all. Thus, if a transaction calls exactly one read-only function of the contract, we will pay the extra one-time cost of loading a cold storage slot. Any further accesses will be cheaper, but still induce the slight extra cost of loading a warm (cached) storage slot. The expected difference in gas costs can be seen in table 6.2.

Cache State	Scenario	Standard	Ours	Delta
Warm	Default	300	300	0
	No external calls	300	100	-200
	Read-only	0	100	100
Cold	Default	2300	2300	0
	No external calls	2300	2100	-200
	Read-only	0	2100	2100

Table 6.2: Differing gas costs of reentrancy protection mechanisms

To verify these differences and put them in context, we measured the gas costs of some simple ERC-20 functions in the various test cases. Note that ERC-20 tokens do not usually require reentrancy protection mechanisms, we simply chose them as they are by far the most ubiquitous type of contract, and ERC-20 transfers are the most common kind of transaction.

A 'warm' ERC20 token transfer was measured to cost the following:

- Unguarded: 10'621 gas
- Standard guard: 10'977 gas
- Our guard: 10'843 gas

As we can see, we are saving 134 gas on this call, out of the expected 200. This is mostly due to the extra arithmetic operations in our reentrancy guard, since it must extract the bit

associated with the specific entrypoint before checking it. It may also be the case that the Solidity compiler is not able to optimize the stack operations as much, since we are restoring the original value of the guard once the call terminates. Thus, any internal function call must clear the stack when it exits, rather than being able to directly return (discarding anything left on the stack for free).

A 'cold' ERC20 total supply check (read-only) was measured to cost:

- Unguarded: 2248 gas
- Our guard: 4489 gas

Here we can see the real cost of loading an extra storage value. Not only are we paying the cost for the runtime check that is usually omitted, we also pay for loading a storage slot that would not otherwise have been accessed. This results in the 2241 extra gas in our call. Note that it is not possible for the `totalSupply` storage slot to be warm while the reentrancy guard is cold, since our approach accesses the guard on every call. Additionally, the base gas cost of a transaction is 21'000 gas. Thus, even if we assume a contract function which costs nothing and instantly returns, the absolute worst case scenario, we pay at most 10.7% more gas. While this does seem expensive, it can occur at most once per transaction, and most transactions are significantly more expensive. However, we do acknowledge that there is a tradeoff at play – is it worth the extra security guarantees to pay a small additional cost on every transaction? As we have seen, read-only reentrancy vulnerabilities have lead to at least $9.6 million dollars in lost funds, if not more. Assuming a conversion of 20'000 gas to $\approx$ $0.01, this would be enough to pay for billions of extra cold storage loads. Hence, we argue that the tradeoff is worth it as default behaviour, although developers might want to opt out of these extra checks in favor of cheaper transactions.

6.2 Effect Annotations

The primary goal of the effect system is to enhance auditability of code. However, a secondary advantage is that developers are required to double-check their work. If they write annotations, but the code induces different effects, this will be reported by the compiler. The compiler can even produce warnings when effects are annotated, but not produced by the code! Thus, any unexpected or missing side effects will be flagged, letting the developer know that something is awry. Furthermore, the tedious nature of writing such annotations will incentivize an approach to development which reduces the number and diversity of side effects. This is a desirable outcome, since this necessarily entails a smaller potential for unintended interactions between effects. Since the effects become simpler to reason about, this will also accelerate the auditing process.

For auditors, the explicit nature of these annotations has two further advantages: As mentioned, they demonstrate the developers intent when writing the code – thus also serving as a form of documentation for the contract under audit. More importantly, the modularity of the effects, i.e. that they are individually declared for each function, allows the auditor to focus on reasoning about a single function at a time, rather than first having to consider which side effects may occur in nested calls.

The clear downside of requiring all side effects to be annotated, of course, is the extra time it takes to write these annotations. Unfortunately, auto-generating these annotations would defeat some of the purpose – the developer would not be double-checking their work, and likely would just ignore the annotations entirely. For auditors, it would still

be useful, although no more useful than the output of any static analysis tool, with the exception of reentrant calls. Due to the native modelling of reentrancy, our compiler is able to directly propagate side effects (or lack thereof) across reentrant calls. This is not possible with any other language, without first running sophisticated static analysis to determine which reentrant calls are even possible.

The key question for developers is of course the following: How many extra lines of code do they need to write? This of course depends heavily on the code architecture. However, we will show that many well-known projects already follow an "effect-oriented" approach to development, meaning there is little additional effort to write annotations, let alone design the code in such a way to minimize their number. For the analysis of the number of additional lines of code, we will use 4 real-world smart contracts, with various levels of complexity:

- ERC-20 Token: 50 LoC
- Simple Token Exchange: 300 LoC
- AAVE Lending Pool Core: 1100 LoC
- Curve Tricrypto: 1500 LoC

For each contract, we count the number of state-modifying functions, as well as the total number of indirect side effects within those function bodies. This lets us estimate the total number of required annotations, and hence give us an approximate overhead when compared to the contract's total lines of code. Of course, this does not necessarily measure the extra *time* required to write the correct annotations, since that might require careful consideration. Still, it gives us an approximate fraction of additional effort that would be necessary due to the effect system. The results of these measurements are detailed in table 6.3.

Contract	State Vars	LoC	Effectful functions	Indirect Effects	Additional LoC
ERC-20 Token	3	50	6 / 9	2	16.67%
Simple Token Exchange	8	300	7 / 16	2	9%
AAVE Lending Pool	5	1100	42 / 88	32	6.73%
Curve Tricrypto	24	1500	33 / 90	24	3.8%

Table 6.3: Additional lines of code required by effect annotations

What we can clearly see is that there is an inverse proportionality relationship between code size and overhead percentage – the larger a contract becomes, the less the annotations contribute to its overall size in terms of lines of code. Complex contracts don't necessarily have many different state variables that are modified in many different combinations – instead, they are often grouped together and modified in similar ways. Therefore, the code dealing with state modifications does not scale in line with the complexity of the contract, instead it grows at a slower rate. Of course, the sample size here is not very significant, it merely serves to indicate the approximate percentages and the inverse relationship.

6.3 Algebraic Data Types

When modelling a state machine, it is standard practice to make use of regular enumeration types. However, such a value can only encode a single state, but not contain any auxiliary information. Standard workarounds include using "magic values", i.e., encoding state information via setting a field to a specific predetermined value, as well as defining a "kitchen-sink" struct, namely one that contains all fields for all states at the same time.

For example, ERC20 tokens are specified such that the value `u256::MAX` $= 2^{256} - 1$ represents an unlimited allowance, whereas any other value represents an allowance equivalent to its actual quantity. Now, any read or write to such an allowance must be checked against this magic value, leading to a higher risk of developer mistakes. Here, we compare the "standard" implementation of the ERC20 `transferFrom` function, as specified by the ERC20 token standard:

Listing 6.1: Solidity

```
contract ERC20 {
  mapping(address => uint256) private _balances;
  mapping(address =>
    mapping(address =>
      uint256)) private _allowances;

  function transferFrom(
    address from,
    address to,
    uint256 value
  ) external returns (bool) {
    require(from != address(0));
    require(to != address(0));

    address spender = msg.sender;
    uint256 amount = _allowances[from][spender];
    if (amount < type(uint256).max) {
      _allowances[from][spender] = amount - value;
    }

    execute_transfer(from, to, value);
    return true;
  }
}
```

Listing 6.2: Ours

```
contract ERC20 {
  balances: HashMap<Address, u256>,
  allowances: HashMap<Address,
    HashMap<Address, Allowance>>,
}
impl ERC20 {
  pub fn transfer_from(self,
    from: Address,
    to: Address,
    value: u256
  ) -> bool {
    assert(from != Address::ZERO);
    assert(to != Address::ZERO);

    let spender = EVM::caller();
    self.allowances[from][spender].spend(value);
    self.execute_transfer(from, to, value);
    return true;
  }
}
enum Allowance {
  Amount(u256),
  Unlimited,
}
impl Allowance {
  fn spend(self, value: u256) {
    match self {
      Allowance::Amount(amount) => {
        amount -= value;
      }
      Allowance::Unlimited => (),
    }
  }
}
```

Note: In both cases, no underflow check is necessary, because both compilers insert overflow checks on arithmetic operations by default. However, it may sometimes be preferable to check manually in order to produce a more specific error message.

Defining an ADT in such a manner provides two distinct advantages: First, it becomes much simpler to separate the underlying logic (reducing the allowance by the spent amount) from the contract's functionality (transferring tokens). Second, the type system now makes it impossible to misinterpret an unlimited allowance as a finite one, whereas in Solidity the developer needs to actively remember to check for the "magic value" every time they use an allowance.

Of course, in such a simple example the benefits are dampened by the overhead induced

by the additional code needed to define the ADT. However, in more complex cases the advantage becomes much clearer. Furthermore, smart contracts often model processes that can be easily represented by a state machine. This is the perfect application for an ADT, as we will see in the next example.

6.3.1 Real-world example

The widely-used OpenZeppelin contract library features a `Governor` contract, which models voting processes. Users can submit proposals, each one of which can be in one of eight different states. Depending on the state, a proposal needs to keep track of different values. For instance, while voting is active, it needs to store the deadline for the end of the voting period, as well as the actual vote counts. This data is stored in the `ProposalCore` struct. Precisely which `ProposalState` the proposal is currently in is dictated by the values stored in the fields. The logic of determining this state is implemented by the `state(...)` function.

Listing 6.3: Proposal State Data

```
struct ProposalCore {
  uint48 voteStart;
  uint32 voteDuration;
  bool executed;
  bool canceled;
  uint48 etaSeconds;
}
enum ProposalState {
  Pending,
  Active,
  Canceled,
  Defeated,
  Succeeded,
  Queued,
  Expired,
  Executed
}
```

Listing 6.4: Proposal State Logic

```
function state(uint256 id) returns (ProposalState) {
  if (proposals[id].executed) {
    return ProposalState.Executed;
  }
  if (proposals[id].canceled) {
    return ProposalState.Canceled;
  }

  uint256 start = proposals[id].voteStart;
  uint256 now = clock();
  if (start >= now) {
    return ProposalState.Pending;
  }

  uint256 deadline = start + proposals[id].voteDuration;
  if (deadline >= now) {
    return ProposalState.Active;
  } else if (!quorumReached(id) || !voteSucceeded(id)) {
    return ProposalState.Defeated;
  } else if (proposals[id].etaSeconds == 0) {
    return ProposalState.Succeeded;
  } else {
    return ProposalState.Queued;
  }
}
```

Recall that this is a widely-used *library contract*, not just some one-off project! As you can see, even just determining what state the proposal is in becomes exceedingly complex. Instead, it would be much simpler to describe a proposal starting from its states, then only include the fields relevant to each state. This is precisely what can be done with an ADT. As an additional benefit, the compiler rejects reading field values that are not relevant to the current state. For some additional clarity, here is what such an ADT might look like, in Rust syntax. It also demonstrates well that most fields are irrelevant for most states.

```
enum ProposalCore {
```

```
    Pending {
        voteStart: u64,
    },
    Active {
        voteStart: u64,
        voteDuration: u64,
    },
    Canceled,
    Defeated,
    Succeeded,
    Queued {
        etaSeconds: u64,
    },
    Expired,
    Executed,
}
```

These examples demonstrate that algebraic data types are not merely a theoretical convenience. They provide concrete, practical improvements to both the correctness and clarity of smart contract code, particularly in the common case of state machine modelling.

Chapter 7

Conclusion

This thesis has demonstrated that language design can meaningfully address the main challenges faced by smart contract developers and auditors today. Instead of placing additional burden on developers, we move responsibility to the compiler and type system. Vulnerabilities can be prevented by adopting a safety-by-default policy, while allowing developers to opt in to unsafe behavior on a case-by-case basis. Importantly, developers must be able to opt in to *exactly only the behaviour they require*, nothing more. Further, increasing auditability of code is an important aspect to the language design in this ecosystem, due to the prevalence of third-party audits, especially for well-known projects. The fact that a large proportion of high-profile exploits involved audited code serves to demonstrate that there is significant potential for improvement in this area. Lastly, it is also important to provide developers with a sufficiently expressive set of features to natively model their desired behaviour, such that its type-safety is guaranteed at compile time, as opposed to manual re-implementations which are more prone to errors.

7.1 Summary of Results

As we have seen, the largest class of vulnerabilities that are amenable to language-level solutions are reentrancies. The majority of real-world examples would have been prevented simply by rejecting all reentrant calls by default – this clearly shows a lack of understanding or tooling in the current ecosystem. Furthermore, it makes more sense for the compiler to preempt issues related to EVM intricacies – otherwise developers are expected to know about and proactively prevent vulnerabilities that do not stem from their own code, but rather from EVM-specific behaviour. To solve these issues, we have introduced a system of path-specific reentrancy guards, which rejects all reentrant calls by default. When a developer requires a reentrant call, they can specifically opt in to exactly one reentrant call path. This is a much safer approach when compared to the standard practice of leaving an entrypoint entirely unguarded. As we have demonstrated, the overhead for these extra protections is relatively benign, with the single exception of read-only functions. However, the term "read-only reentrancy" has been coined specifically to describe the vulnerability this is preventing, demonstrating that this is a significant threat which is worth protecting against.

Most importantly, we have clearly demonstrated that our approach to reentrancy protection can effectively prevent real-world vulnerabilities. Of the 54 examples of reentrancies in the dataset, 53 resulted from errors in developer code, and 16 of those projects did actually make use of reentrant calls in some way. Of those 16 vulnerabilities, all of them could have

been prevented by selectively re-enabling only the required functionality. Of course, this does not rule out the potential for developer error – it is still possible for a developer to explicitly allow a reentrant call path that is unsafe. However, given the vast reduction in the number of possibilities there are to reason about, it becomes much easier to find or rule out reentrancy vulnerabilities.

Our second key feature is the introduction of side-effect annotations. It has two key benefits – firstly, it enforces an "effect-oriented" approach to smart contract development, which is already relatively widespread among popular projects. This approach also serves as a mechanism which forces the developer to double-check their work, at least when it comes to critical state modifications. More importantly, the intent is to accelerate the auditing process, as well as enabling auditors to rule out certain interactions more quickly, allowing them to dive deeper into the space of possibilities without worrying about surface-level bugs. It also offloads the responsibility of correctly inferring which effects happen where to the compiler, rather than being prone to human errors. The developer overhead induced by these annotations is somewhat significant for very small contracts, but as contract size grows it becomes more negligible.

Lastly, we have introduced the notion of algebraic data types to the EVM. It is a natural fit for multiple reasons - first of all, the modelling of state machines is a common requirement, and ADTs model these in a natural and type-safe way. Through exhaustive pattern matching, the compiler can also ensure that all possible cases are handled, eliminating the chance for developer error. ADTs such as Rust's `Option` and `Result` allow native handling of failures or errors, which is a major improvement over "silent" failure. Further, combining ADTs with the coding style of idiomatic Rust also naturally separates *pure* functionality from that of the contract itself. In other EVM languages, functions are almost always defined in "contract space", meaning they cannot logically be separated from the underlying contract, even if the functionality is free of side effects. By contrast, we have found it more natural to define methods on the types themselves, meaning the code in "contract space" becomes sparser and more focused on the contract's intended functionality, as opposed to the implementation of arithmetic operations or other more abstract functionalities. This also paves the path for more natural code reuse, since the user-defined types can be fully separated from the smart contracts. Finally, this separation also enables the possibility of unit testing such pure code without the need of mocking a blockchain, contracts, or any such complexities.

7.2 Limitations

The increased complexity of the type system and compiler is concerning – especially given the difficulty of type inference involving generics, method resolution, and further challenges. The two-level hierarchy of compilers further increases the risk of errors – not only do we rely on our own compiler's correctness, but now also on that of the Solidity compiler. Of course, we only rely on relatively simple functionality, and the Solidity compiler is by far the most battle-tested option available. For the moment, the advantages provided by leveraging the tools available for Solidity code far outweigh the risk of compiler errors, but reducing the reliance on external tools does provide a natural way forward.
Regarding the complexity of the type system, it is clear that it does provide additional sources of compiler error. However, reasoning about such abstract types is a well-understood field of study, and Hindley-Milner type inference, as well as the specific approach we have based our inference algorithm on, are known to be sound. The intent is mostly to allow abstractions of functionality over monadic types, for example by defining addition over

Rust's `Option` type. Contract code cannot directly contain generics, only instantiate generic code with concrete types.

7.3 Future Directions

7.3.1 Adoption Road-Blocks

Before the language could be considered production-ready, several engineering milestones remain. Many intrinsic EVM features, such as event logging, delegate-calls, and programmatic contract creation are not yet supported. Further, the only tooling available is due to the reliance on the Solidity compiler, if this were eventually phased out, then tools such as debuggers, testing and deployment frameworks would need to be created. Additionally, the modularity of the language needs to be expanded to allow for libraries, contract composition, and other mechanisms for code reuse.

7.3.2 Compiler Work

A natural way forward is to reduce the reliance on the Solidity compiler. This can be done incrementally without any problem, since one could smoothly transition from generating native Solidity to including more and more inline assembly. Note that "inline assembly" in Solidity really just means inline Yul, which is the intermediate representation used by the compiler. Once the generated code consists only of this inline assembly, it can be easily transitioned to generate full Yul contracts instead. This would still benefit from the Solidity compiler's powerful optimizer stage, which operates on the Yul IR directly – thus having comparable performance to Solidity.

Interpreter

Another interesting avenue would be the implementation of an interpreter for the language. This would provide an easy way of emulating the behaviour of internal functions, without the need for mocking the entire EVM or blockchain. Current debugging and testing tools have to deploy a local test-chain to run the contracts in, which is clearly a huge amount of overhead. While this isn't prohibitive for integration tests, unit testing is too slow for rapid prototyping work, and debugging smart contract code is quite tedious. Moreover, fuzz testing is heavily reliant on performance, and thus being able to directly interpret the code without the overhead of the EVM could provide a significant advantage over existing tools. If such an interpreter could also easily model external contract interactions, it may be possible to eliminate the mocking of the EVM entirely.

7.3.3 User Studies

One important open question is whether the effect system, as currently designed, strikes the right balance between safety and usability. While the annotations provide clear benefits for auditors, they also introduce significant developer overhead, particularly for smaller contracts. User studies would be a natural way to evaluate this tradeoff empirically. For instance, one could study whether auditors are able to find or rule out vulnerabilities more quickly in code written in our language with and without effect annotations, and compared to equivalent Solidity or Vyper code. Similarly, a study of developer experience could assess whether the annotation syntax is intuitive in practice, or whether alternative ways of declaring effects would reduce friction without sacrificing clarity. Such studies would also

provide valuable data on the learning curve associated with adopting new languages in a high-stakes environment, and could directly inform future iterations of the language design. This kind of empirical evaluation is well-established in programming language research [35] [9], especially when considering advanced type systems [19], and would complement the formal and dataset-based evaluation presented in this thesis.

7.3.4 Language Extensions

There are several interesting avenues for future language design research. For one, many systems involve multiple contracts interacting with one another. Because they all belong to the same project, the calls between the different contracts are trusted. As such, the effect system could be extended to model not only the current contract's storage, but also that of other known contracts. Furthermore, reentrancy guards could be omitted on one contract if it can only ever be accessed through another, leading to substantial gas savings. In a similar vein, many contracts end up being split into multiple parts, due to the size restriction imposed on contract bytecode. This could be done automatically by the compiler, perhaps even optimizing for the fewest number of cross-contract calls or some other cost metric, instead of relying on developers to figure out their own workarounds.

Access control is another aspect of smart contracts that can be improved by language design – many experimental languages introduce their own novel access control primitives. Even still, access control bugs result in a significant number of exploits, again demonstrating significant potential for improvement. However, it is difficult to define unifying principles that are shared between the large variety of access control mechanisms used by smart contracts in practice.

In the last year or two, there has been a recent trend of exploits taking advantage of precision loss, even in audited projects. This suggests that existing tools provide insufficient support for reasoning about rounding errors. While it may be difficult for this to be addressed directly with the design of a language, perhaps other approaches such as static analysis could allow checking safe bounds on computations involving rounding errors.

Lastly, once the type and effect systems have been stabilized, it becomes important to ensure their correctness. This could be done for instance by formally verifying their implementations, providing a foundation for a formally verified compiler stack.

7.4 Closing Remarks

The prevalence of reentrancy vulnerabilities is not an unavoidable consequence of the EVM, but a lack of adequate protection mechanisms in available tools. Furthermore, our language design in principle allows static analysis (or similar reasoning) to propagate across reentrant calls, which is not natively possible in any other existing languages. It also improves reasoning capabilities regarding side effects, which may help auditors to find or rule out vulnerabilities more easily. The coding style incentivized by the language also seems to naturally separate effectful code from more pure, abstract functionalities. We have also prioritized expressiveness in the type system to offload more responsibility to the compiler, so that developers and auditors can focus more on the intended functionality rather than worrying about compiler or EVM intricacies.

The smart contract ecosystem has long treated security as an afterthought, relying on developer discipline and post-hoc auditing and analysis to catch vulnerabilities that should never

have been possible in the first place. This thesis has argued, and empirically demonstrated, that language design offers a more principled path forward – one where safety guarantees are enforced by construction rather than convention. While significant work remains before such a language could see widespread adoption, the convergence of independent efforts such as Fe's recent redesign toward similar principles suggests that the ecosystem is beginning to recognise this. The question is no longer whether language design can improve smart contract security, but how far that improvement can go.

Bibliography

[1] Patrick Bahr, Jost Berthold, and Martin Elsman. Certified symbolic management of financial multi-party contracts. In *Proceedings of the 20th ACM SIGPLAN International Conference on Functional Programming*, ICFP 2015, page 315–327, New York, NY, USA, 2015. Association for Computing Machinery.

[2] Sam Blackshear, Evan Cheng, David L. Dill, Victor Gao, Ben Maurer, Todd Nowacki, Alistair Pott, Shaz Qadeer, Dario Rain, Stephane Russi, Tim Sezer, Runtian Zakian, and Zhou. Move: A language with programmable resources. 2019.

[3] EDWIN BRADY. Idris, a general-purpose dependently typed programming language: Design and implementation. *Journal of Functional Programming*, 23(5):552–593, 2013.

[4] Edwin Brady. Idris 2: Quantitative Type Theory in Practice. In Anders Møller and Manu Sridharan, editors, *35th European Conference on Object-Oriented Programming (ECOOP 2021)*, volume 194 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 9:1–9:26, Dagstuhl, Germany, 2021. Schloss Dagstuhl – Leibniz-Zentrum für Informatik.

[5] Chainalysis. The chainalysis 2026 crypto crime report. https://www.chainalysis.com/reports/crypto-crime-2026/, 2024. Accessed: 2026-04-07.

[6] ChainSecurity. Curve lp oracle manipulation: Post mortem. https://www.chainsecurity.com/blog/curve-lp-oracle-manipulation-post-mortem, 2022.

[7] Stefanos Chaliasos, Marcos Antonios Charalambous, Liyi Zhou, Rafaila Galanopoulou, Arthur Gervais, Dimitris Mitropoulos, and Benjamin Livshits. Smart contract and defi security tools: Do they meet the needs of practitioners? In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, ICSE '24, New York, NY, USA, 2024. Association for Computing Machinery.

[8] ConsenSys Diligence. Mythril: Security analysis tool for evm bytecode. https://github.com/ConsenSysDiligence/mythril, 2018. Accessed: 2026-04-07.

[9] Will Crichton. The usability of ownership, 2021.

[10] Luis Damas and Robin Milner. Principal type-schemes for functional programs. In *Proceedings of the 9th ACM Symposium on Principles of Programming Languages*, pages 207–212, 1982.

[11] DeFiHackLabs. Reproduce defi hacks using foundry. https://github.com/SunWeb3Sec/DeFiHackLabs, 2026. Accessed: 2026-04-07.

[12] Benjamin Egelund-Müller, Martin Elsman, Fritz Henglein, and Omri Ross. Automated execution of financial contracts on blockchains. *Business & Information Systems Engineering*, 59(6):457–467, 2017.

[13] Ethereum Foundation. Hard fork completed. https://blog.ethereum.org/2016/07/20/hard-fork-completed, 2016.

[14] Ethereum Foundation. Fe programming language. https://fe-lang.org, 2020. Accessed: 2026-04-07.

[15] Ethereum Foundation. Solidity documentation: Security considerations. https://docs.soliditylang.org/en/latest/security-considerations.html, 2024.

[16] Ethereum Foundation. Fe programming language. https://blog.fe-lang.org/posts/fe26-a-fresh-start/, 2026. Accessed: 2026-04-07.

[17] Ethereum Foundation. Solidity documentation. https://docs.soliditylang.org, 2026. Accessed: 2026-04-07.

[18] Josselin Feist, Gustavo Grieco, and Alex Groce. Slither: A static analysis framework for smart contracts. In *2019 IEEE/ACM 2nd International Workshop on Emerging Trends in Software Engineering for Blockchain (WETSEB)*, pages 8–15, 2019.

[19] Kasra Ferdowsi. The usability of advanced type systems: Rust as a case study, 2023.

[20] Franziska Heintel & USC Judges. Announcing the winners of the underhanded solidity contest 2022. https://www.soliditylang.org/blog/2022/04/08/announcing-the-underhanded-contest-winners-2022/, 2022. Accessed: 2026-04-07.

[21] Ioannis Gabrielides. Automatic verification of flint smart contracts. Master's thesis, Imperial College London, 2019. Available at https://github.com/flintlang/flint/blob/master/docs/pdfs%20(student%20theses)/verification/IoannisGabrielides.pdf, Accessed: 2026-04-07.

[22] Asem Ghaleb and Karthik Pattabiraman. How effective are smart contract analysis tools? evaluating smart contract static analysis tools using bug injection. In *Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis*, ISSTA 2020, page 415–427, New York, NY, USA, 2020. Association for Computing Machinery.

[23] J. Roger Hindley. The principal type-scheme of an object in combinatory logic. *Transactions of the American Mathematical Society*, 146:29–60, 1969.

[24] Immunefi. Crypto losses in 2024. https://immunefi.com/research/, 2024. Accessed: 2026-04-07.

[25] Mark Jones. Typing haskell in haskell. 03 2001. Accessed: 2026-04-07.

[26] Matter Labs. Zinc framework. https://github.com/matter-labs/zinc, 2020. Accessed: 2026-04-07.

[27] Robin Milner. A theory of type polymorphism in programming. *Journal of Computer and System Sciences*, 17(3):348–375, 1978.

[28] Nadcab Labs. Choosing between solidity and vyper for icos- a professional developer's perspective. https://www.nadcab.com/blog/solidity-vs-vyper-ico, 2026. Accessed: 2026-04-07.

[29] OpenZeppelin. Openzeppelin contracts. https://docs.openzeppelin.com/contracts, 2026. Accessed: 2026-04-07.

[30] Jack Pettersson and Robert Edström. Safer smart contracts through type-driven development. 2016.

[31] Claudia Ruggiero, Pietro Mazzini, Emilio Coppa, Simone Lenti, and Silvia Bonomi. Sok: A unified data model for smart contract vulnerability taxonomies. In *Proceedings of the 19th International Conference on Availability, Reliability and Security*, ARES '24, New York, NY, USA, 2024. Association for Computing Machinery.

[32] Franklin Schrans, Susan Eisenbach, and Sophia Drossopoulou. Writing safe smart contracts in flint. In *Companion Proceedings of the 2nd International Conference on the Art, Science, and Engineering of Programming*, Programming '18, page 218–219, New York, NY, USA, 2018. Association for Computing Machinery.

[33] Ilya Sergey, Amrit Kumar, and Aquinas Hobor. Scilla: a smart contract intermediate-level language, 2018.

[34] SlowMist. Blockchain security and anti-money laundering annual report. https://www.slowmist.com/report/2024-Blockchain-Security-and-AML-Annual-Report(EN).pdf, 2024. Accessed: 2026-04-07.

[35] Andreas Stefik and Susanna Siebert. An empirical investigation into programming language syntax. *ACM Trans. Comput. Educ.*, 13(4), November 2013.

[36] Vyper Team. Vyper documentation. https://docs.vyperlang.org, 2026. Accessed: 2026-04-07.

[37] Gavin Wood. Ethereum: A secure decentralised generalised transaction ledger. https://ethereum.github.io/yellowpaper/paper.pdf, 2014. Accessed: 2026-04-07.

Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

Declaration of originality

The signed declaration of originality is a component of every written paper or thesis authored during the course of studies. In consultation with the supervisor, one of the following three options must be selected:

- ○ I confirm that I authored the work in question independently and in my own words, i.e. that no one helped me to author it. Suggestions from the supervisor regarding language and content are excepted. I used no generative artificial intelligence technologies[1].
- ○ I confirm that I authored the work in question independently and in my own words, i.e. that no one helped me to author it. Suggestions from the supervisor regarding language and content are excepted. I used and cited generative artificial intelligence technologies[2].
- ● I confirm that I authored the work in question independently and in my own words, i.e. that no one helped me to author it. Suggestions from the supervisor regarding language and content are excepted. I used generative artificial intelligence technologies[3]. In consultation with the supervisor, I did not cite them.

Title of paper or thesis:

Taming Reentrancy with Auditable Language Design

Authored by:
If the work was compiled in a group, the names of all authors are required.

Last name(s):	First name(s):
Richards	Tynan

With my signature I confirm the following:
- I have adhered to the rules set out in the Citation Guide.
- I have documented all methods, data and processes truthfully and fully.
- I have mentioned all persons who were significant facilitators of the work.

I am aware that the work may be screened electronically for originality.

Place, date	Signature(s)
Dietikon, 07.04.2026	T Richards

If the work was compiled in a group, the names of all authors are required. Through their signatures they vouch jointly for the entire content of the written work.

[1] E.g. ChatGPT, DALL E 2, Google Bard
[2] E.g. ChatGPT, DALL E 2, Google Bard
[3] E.g. ChatGPT, DALL E 2, Google Bard